Workshop Https Certificate

Esilv 2026

Outline

- Part 1 : Https Certificate on Client-Side
  - A/ using Web Browser
  - B/ using Command Line
- Part 2 : Trusted Certificates
  - Trusted on OS-system (Chrome -> in Windows)
  - Trusted by application, ex: java
- Part 3 : Https Certificate on Server-Side
  - Generating a Self-signed certificate (KeyStore Explorer UI)
  - Generating a Self-signed certificate (command line)
  - Certificate Chain & Issuer
  - Exposing https in springboot web server

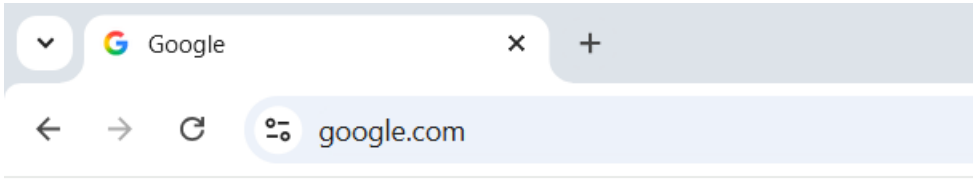
Part 1 … Https Certificate on Client-Side (using Web Browser)

Open a Https:// web site

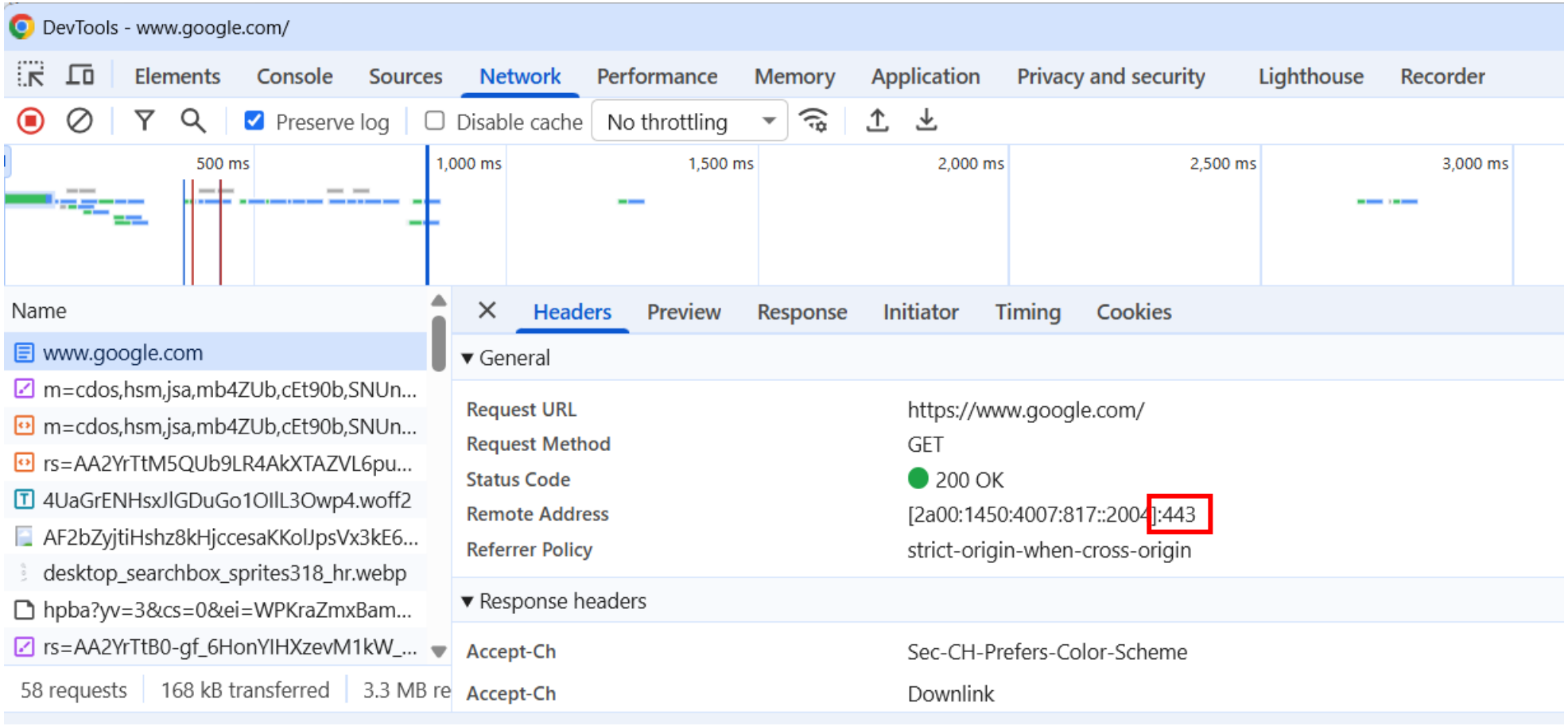
example: https://www.google.com

(equivalent to https://www.google.com:443/)

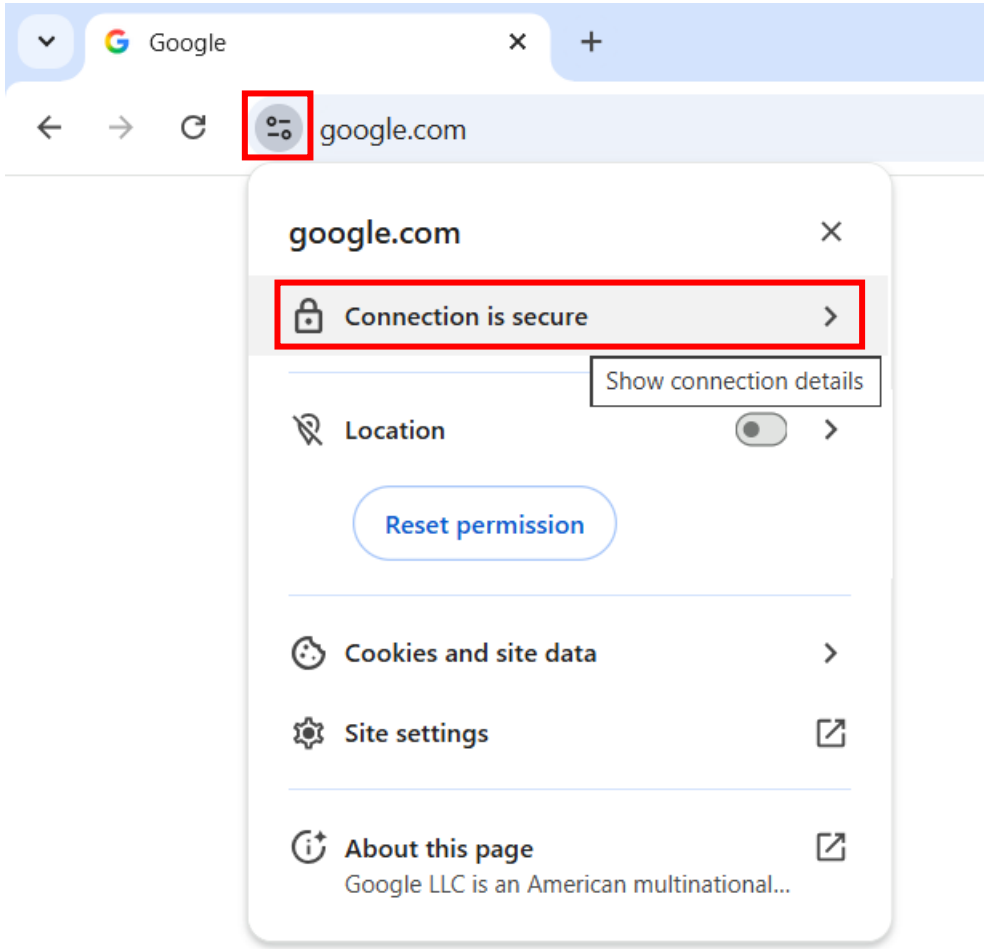
Other example: https://my.devinci.fr/

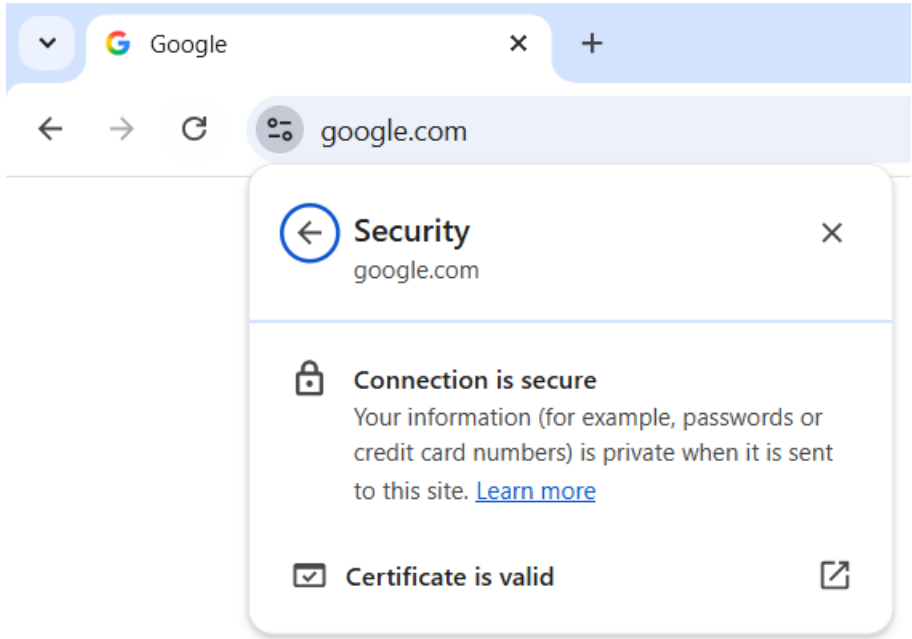


Using ChromeDev tool, check the TCP port is 443 (http => 80, or 8080 / https => 443)

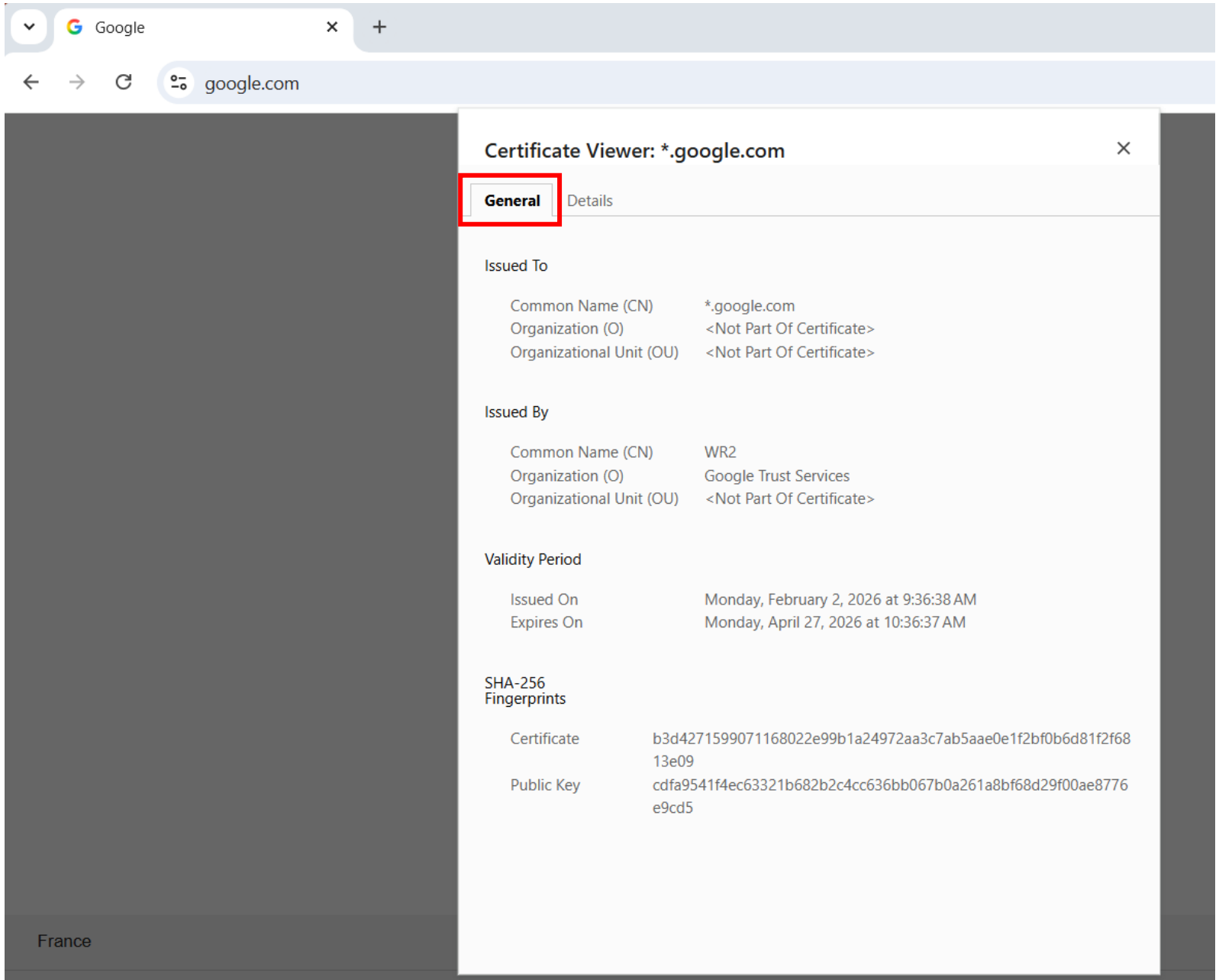


View the Certificate

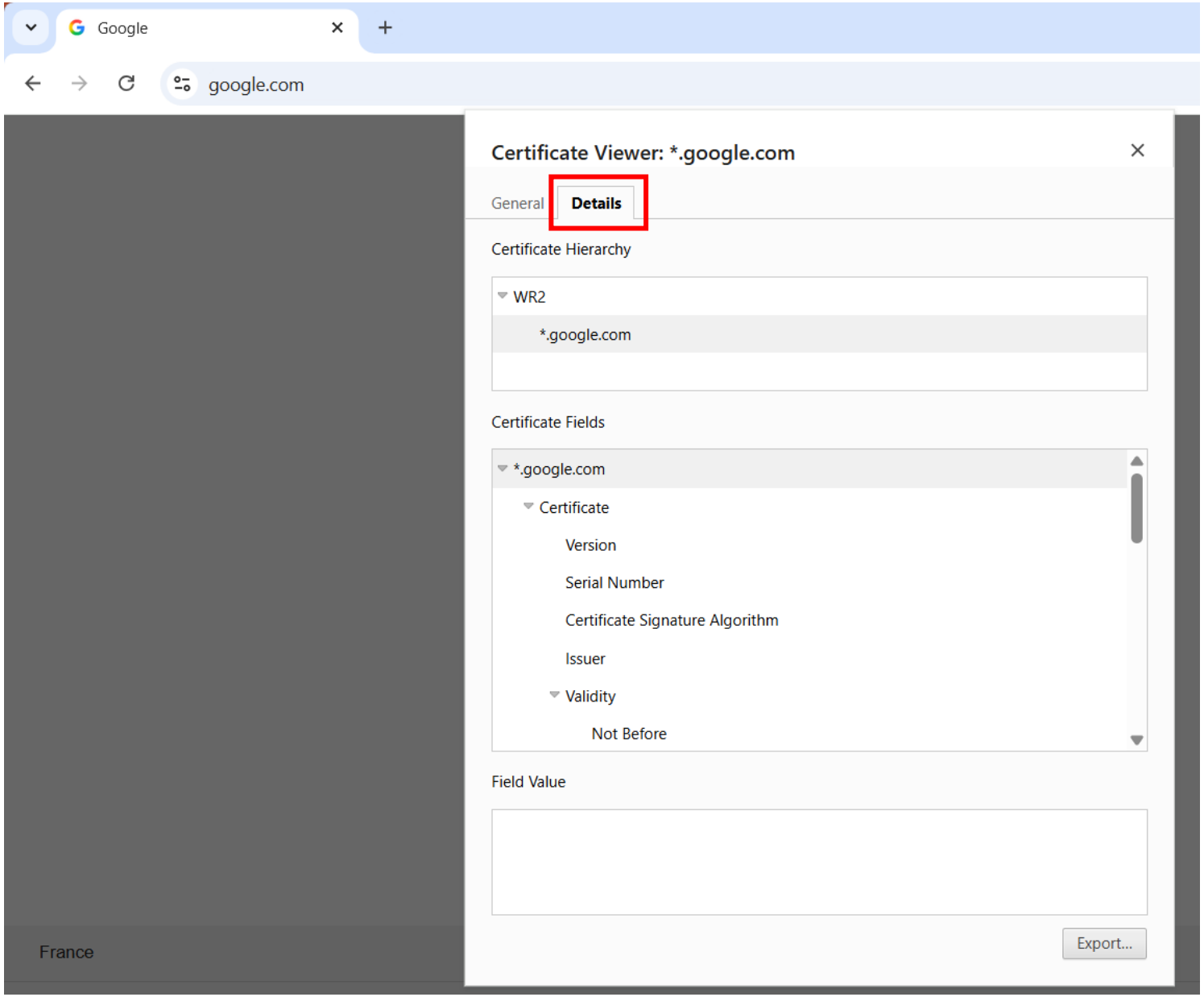




Certificate View (Tab: General)

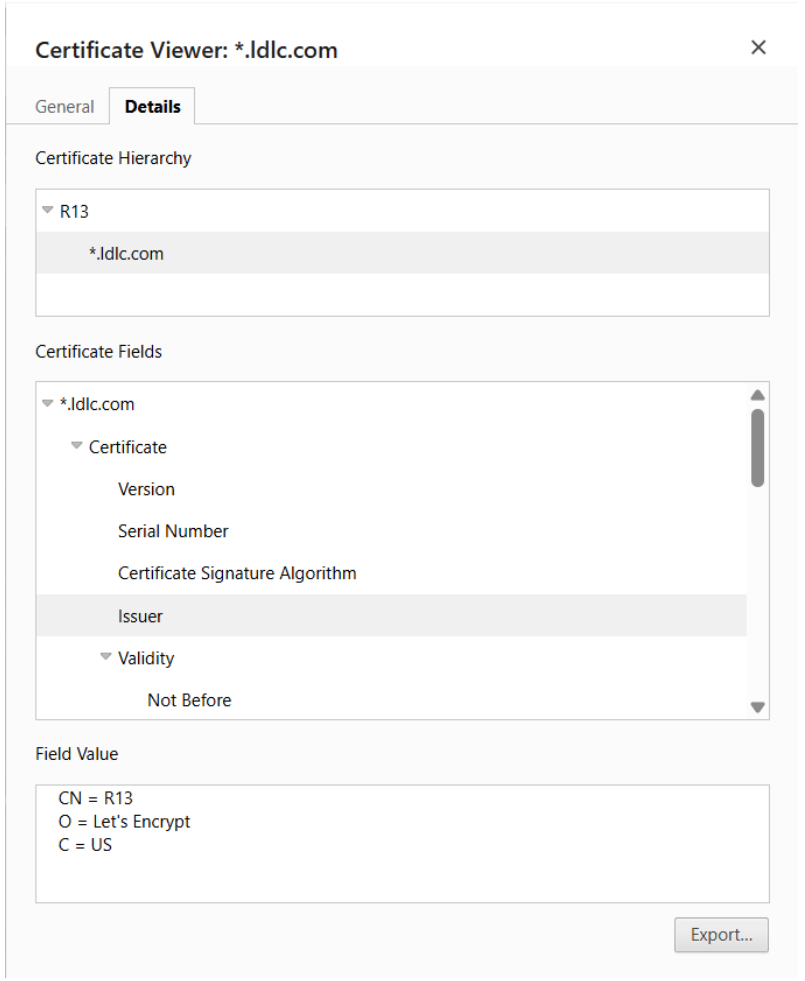


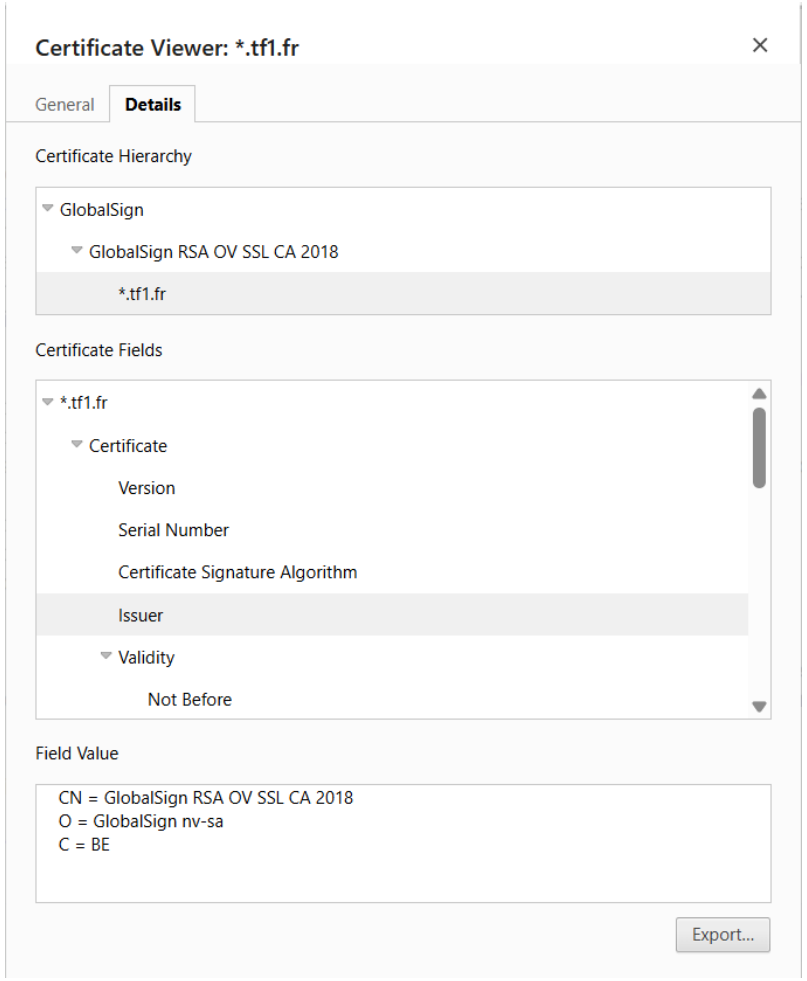
Certificate Details



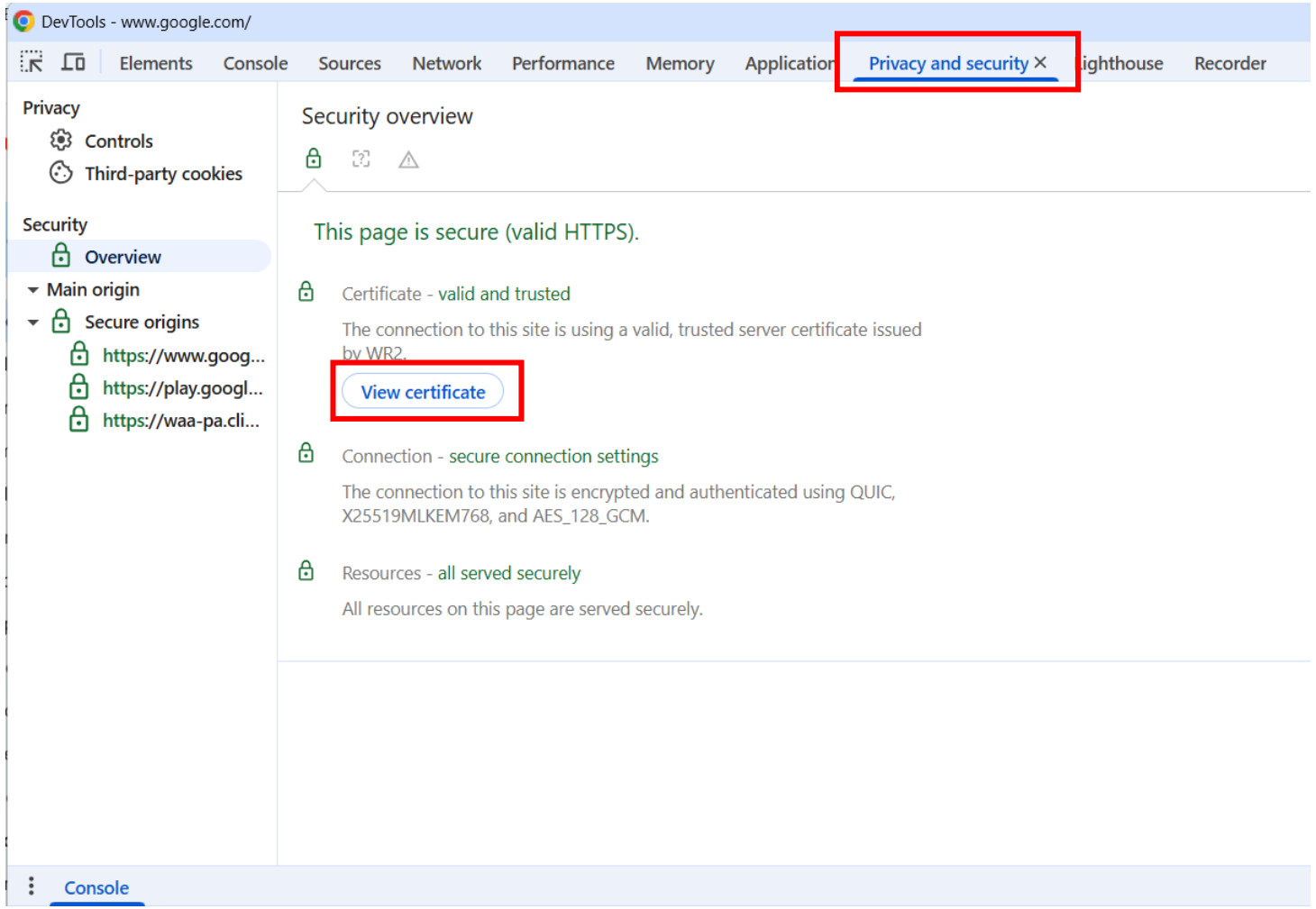
Certificate Hierarchy

Look at other web sites… Hierarchy of 1 / 2 / 3 Issuers
Example of Issuers: « Google Trust Services », « Let's Encrypt », « GlobalSign RSA », …

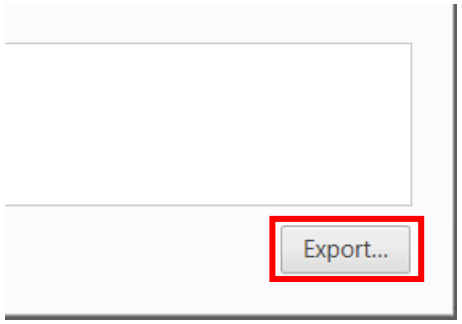


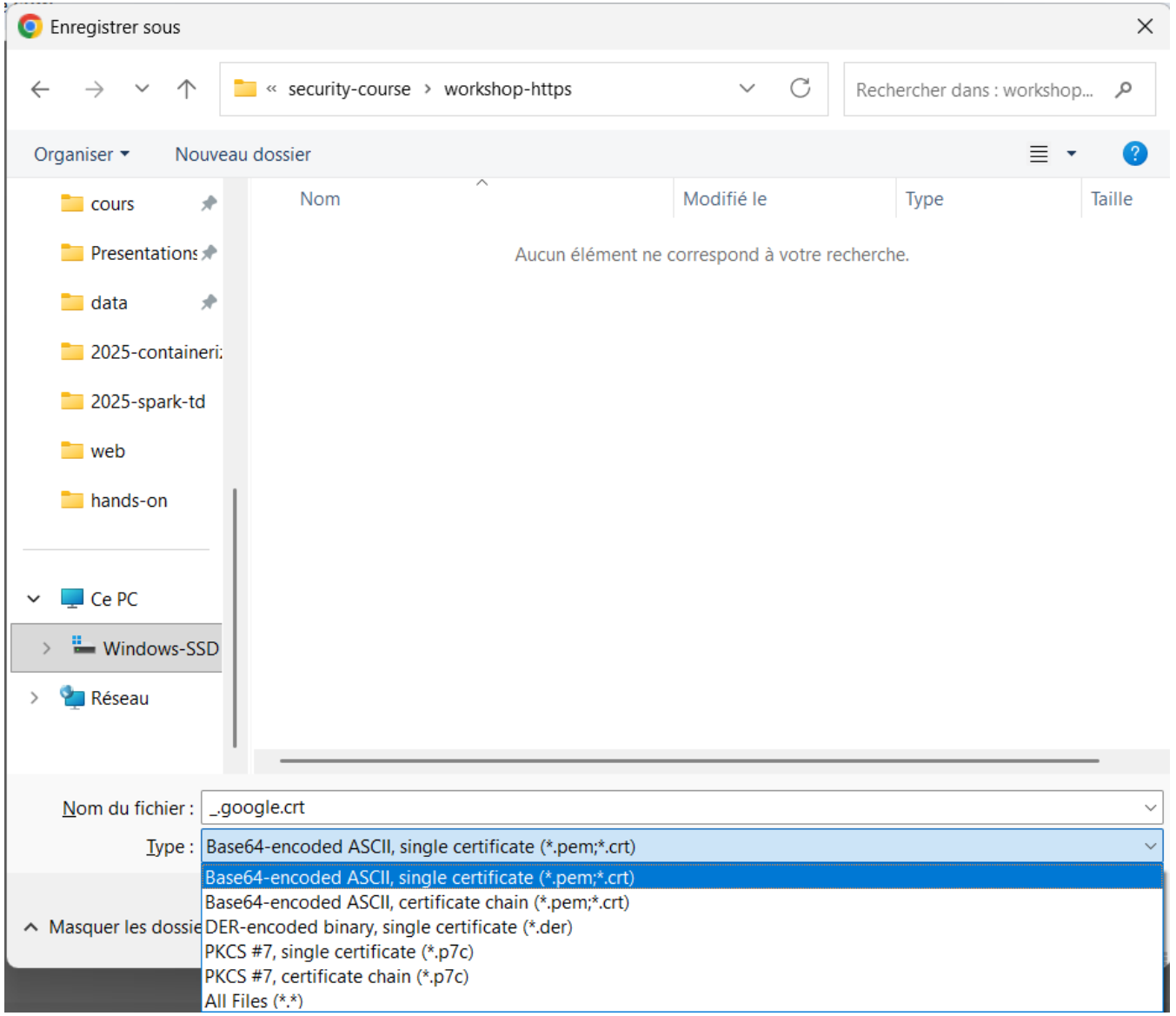


Remark: Can also view certificate from DevTool > Privacy and security > View Certificate



Export the Certificate as a single certificate *.pem file

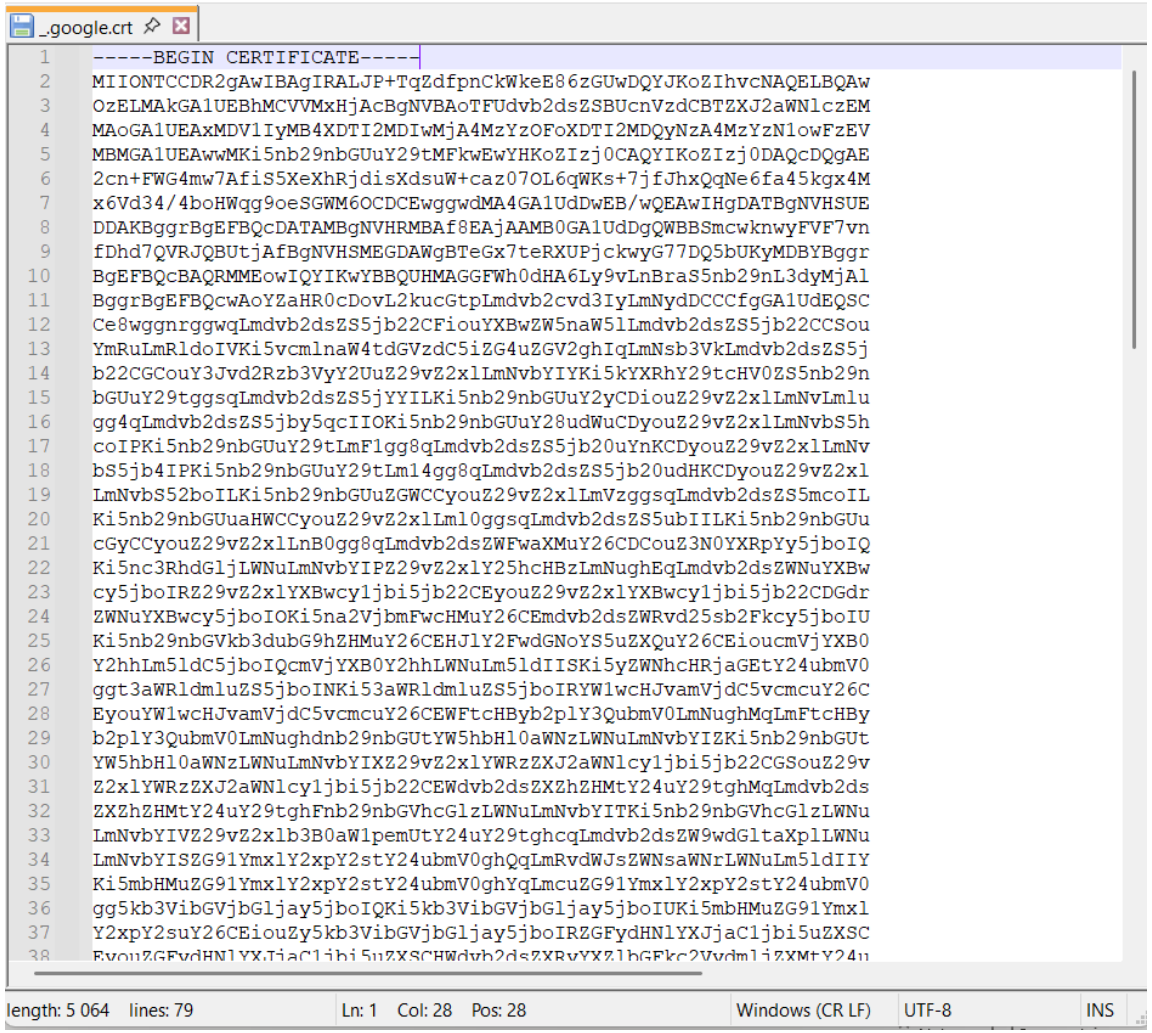




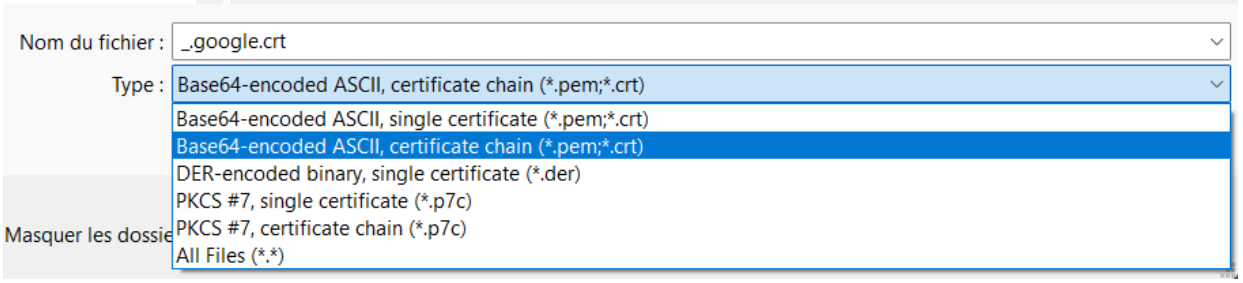
View (base64 = text file)

Check that it is a single section of text, wrapped with

-----BEGIN CERTIFICATE-----
<<base64>>
-----END CERTIFICATE-----

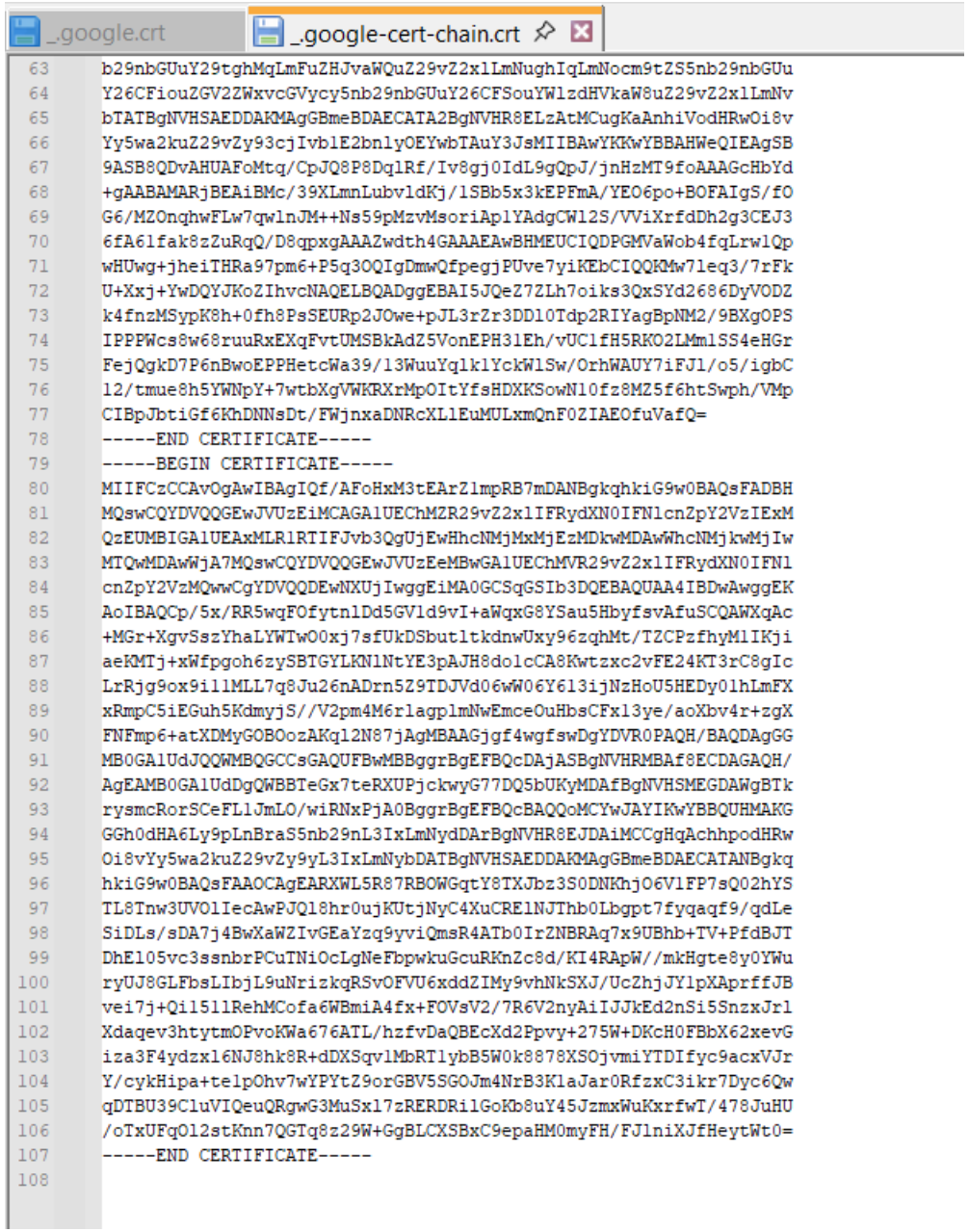


Compare with full certificate Chain



Check that it is
(written is reverse order compared to UI)

-----BEGIN CERTIFICATE-----
<<base64>>
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
<<base64>> (parent… root certificate)
-----END CERTIFICATE-----



Part 1 b/ … Https Certificate on Client-Side (using Shell Commands Line)

Https connection from Shell Command Line(curl)

Cmder

```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -v https://www.google.com
* Host www.google.com:443 was resolved.
* IPv6: 2a00:1450:4007:80f::2004
* IPv4: 142.251.142.4
*   Trying [2a00:1450:4007:80f::2004]:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to www.google.com (2a00:1450:4007:80f::2004 port 443) from 2a01:e0a:1f7:1940:18da:b269:a2e9:36f
7 port 57848
* using HTTP/1.x
> GET / HTTP/1.1
> Host: www.google.com
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 200 OK
< Date: Sat, 07 Mar 2026 10:25:41 GMT
< Expires: -1
```

Https From Curl -vv

curl –vv https://www.google.com

```
10:27:07.638000 [0-0] * Host www.google.com:443 was resolved.
10:27:07.654000 [0-0] * IPv6: 2a00:1450:4007:80f::2004
10:27:07.654000 [0-0] * IPv4: 142.251.142.4
10:27:07.662000 [0-0] * [HTTPS-CONNECT] adding wanted h2
10:27:07.662000 [0-0] * [HTTPS-CONNECT] added
10:27:07.662000 [0-0] * [HTTPS-CONNECT] connect, init
10:27:07.669000 [0-0] *   Trying [2a00:1450:4007:80f::2004]:443...
10:27:07.669000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
10:27:07.669000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
10:27:07.669000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 1 socks
10:27:07.685000 [0-0] * schannel: disabled automatic use of client certificate
10:27:07.693000 [0-0] * ALPN: curl offers http/1.1
10:27:07.693000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
10:27:07.701000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
10:27:07.706000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 1 socks
10:27:07.708000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=0
10:27:07.708000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=0
10:27:07.717000 [0-0] * [HTTPS-CONNECT] adjust_pollset -> 0, 1 socks
10:27:07.717000 [0-0] * ALPN: server accepted http/1.1
10:27:07.717000 [0-0] * [HTTPS-CONNECT] connect+handshake h2: 60ms, 1st data: 40ms
10:27:07.733000 [0-0] * [HTTPS-CONNECT] connect -> 0, done=1
10:27:07.733000 [0-0] * [HTTPS-CONNECT] Curl_conn_connect(block=0) -> 0, done=1
10:27:07.733000 [0-0] * Established connection to www.google.com (2a00:1450:4007:80f::2004 port 443) from
2a01:e0a:1f7:1940:18da:b269:a2e9:36f7 port 57855
10:27:07.749000 [0-0] * [HTTPS-CONNECT] query ALPN
10:27:07.749000 [0-0] * using HTTP/1.x
10:27:07.749000 [0-0] > GET / HTTP/1.1
10:27:07.749000 [0-0] > Host: www.google.com
```

Certificate From OpenSSL command

openssl s_client -connect www.google.com:443 -servername www.google.com

Certificate chain has 3 levels ??
We saw 2 in Chrome ??

Cmder

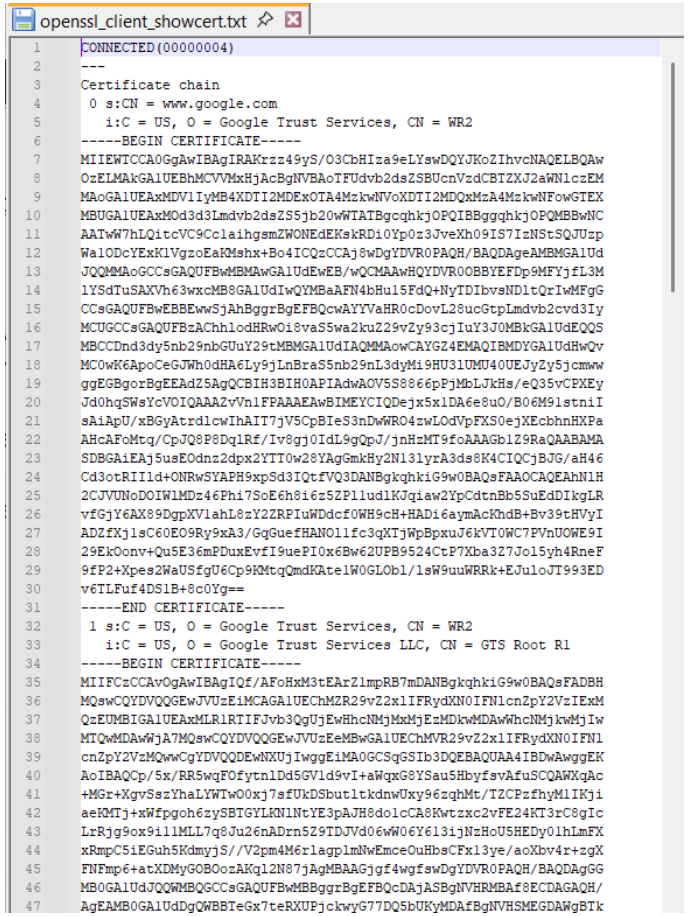
```
$ openssl s_client -connect www.google.com:443 -servername www.google.com
CONNECTED(00000004)
depth=2 C = US, O = Google Trust Services LLC, CN = GTS Root R1
verify return:1
depth=1 C = US, O = Google Trust Services, CN = WR2
verify return:1
depth=0 CN = www.google.com
verify return:1
---
Certificate chain
 0 s:CN = www.google.com
   i:C = US, O = Google Trust Services, CN = WR2
 1 s:C = US, O = Google Trust Services, CN = WR2
   i:C = US, O = Google Trust Services LLC, CN = GTS Root R1
 2 s:C = US, O = Google Trust Services LLC, CN = GTS Root R1
   i:C = BE, O = GlobalSign nv-sa, OU = Root CA, CN = GlobalSign Root CA
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIEWTCCA0GgAwIBAgIRAKrzz49yS/O3CbHIza9eLYswDQYJKoZIhvcNAQELBQAw
OzELMAkGA1UEBhMCVVMxHjAcBgNVBAoTFUdvb2dsZSBUcnVzdCBTZXJ2aWNlczEM
MAoGA1UEAxMDV1IyMB4XDTI2MDExOTA4MzkwNVoXDTI2MDQxMzA4MzkwNFowGTEX
MBUGA1UEAxMOd3d3Lmdvb2dsZS5jb20wWTATBgcqhkjOPQIBBggqhkjOPQMBBwNC
AATwW7hLQitcVC9Cc1aihgsmZWONEdEKskRDi0Yp0z3JveXh09IS7IzNStSQJUzp
Wa1ODcYExK1VgzoEaKMshx+Bo4ICQzCCAj8wDgYDVR0PAQH/BAQDAgeAMBMGA1Ud
JQQMMAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVR0OBBYEFDp9MFYjfL3M
```

Openssl … see also option « -showcerts »

echo | openssl s_client -connect www.google.com:443 -servername www.google.com **-showcerts**

$ echo | openssl s_client -connect www.google.com:443 -servername www.google.com -showcerts \
  > /cygdrive/c/security-course/workshop-https/openssl_client_showcert.txt

depth=2 C = US, O = Google Trust Services LLC, CN = GTS Root R1
verify return:1
depth=1 C = US, O = Google Trust Services, CN = WR2
verify return:1
depth=0 CN = www.google.com
verify return:1
DONE


openssl_client_showcert.txt

```
CONNECTED(00000004)
---
Certificate chain
 0 s:CN = www.google.com
   i:C = US, O = Google Trust Services, CN = WR2
-----BEGIN CERTIFICATE-----
MIIEWTCCA0GgAwIBAgIRAKrzz49yS/O3CbHIza9eLYswDQYJKoZIhvcNAQELBQAw
OzELMAkGA1UEBhMCVVMxHjAcBgNVBAoTFUdvb2dsZSBUcnVzdCBTZXJ2aWNlczEM
MAoGA1UEAxMDV1IyMB4XDTI2MDExOTA4MzkwNVoXDTI2MDQxMzA4MzkwNFowGTEX
MBUGA1UEAxMOd3d3Lmdvb2dsZS5jb20wWTATBgcqhkjOPQIBBggqhkjOPQMBBwNC
AATwW7hLQitcVC9CclaihgsmZWONEdEKskRDiOYp0z3JveXh09IS7IzNStSQJUzp
Wa1ODcYExKlVgzoEaKMshx+Bo4ICQzCCAj8wDgYDVR0PAQH/BAQDAgeAMBMGA1Ud
JQQMMAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVR0OBBYEFDp9MFYjfL3M
lYSdTuSAXVh63wxcMB8GA1UdIwQYMBaAFN4bHul5FdQ+NyTDIbvsNDltQrIwMFgG
CCsGAQUFBwEBBEwwSjAhBggrBgEFBQcwAYYVaHR0cDovL28ucGtpLmdvb2cvd3Iy
MCUGCCsGAQUFBzAChhlodHRwOi8vaS5wa2kuZ29vZy93cjIuY3J0MBkGA1UdEQQS
MBCCDnd3dy5nb29nbGUuY29tMBMGA1UdIAQMMAowCAYGZ4EMAQIBMDYGA1UdHwQv
MC0wK6ApoCeGJWh0dHA6Ly9jLnBraS5nb29nL3dyMi9HU3lUMU40UEJyZy5jcmww
ggEGBgorBgEEAdZ5AgQCBIH3BIH0APIAdwAOV5S8866pPjMbLJkHs/eQ35vCPXEy
Jd0hqSWsYcVOIQAAAZvVnlFPAAAEAwBIMEYCIQDejx5xlDA6e8uO/B06M9lstniI
sAiApU/xBGyAtrdlcwIhAIT7jV5CpBIeS3nDwWRO4zwLOdVpFXS0ejXEcbhnHXPa
AHcAFoMtq/CpJQ8P8DqlRf/Iv8gj0IdL9gQpJ/jnHzMT9foAAAGblZ9RaQAABAMA
SDBGAiEAj5usEOdnz2dpx2YTT0w28YAgGmkHy2N13lyrA3ds8K4CIQCjBJG/aH46
Cd3otRIIld+ONRwSYAPH9xpSd3IQtfVQ3DANBgkqhkiG9w0BAQsFAAOCAQEAhNlH
2CJVUNoDOIWlMDz46Phi7SoE6h8i6z5ZPlludlKJqiaw2YpCdtnBb5SuEdDIkgLR
vfGjY6AX89DgpXV1ahL8zY2ZRPIuWDdcf0WH9cH+HADi6aymAcKhdB+Bv39tHVyI
ADZfXjlsC60EO9Ry9xA3/GqGuefHANOl1fc3qXTjWpBpxuJ6kVT0WC7PVnUOWE9I
29Ek0onv+Qu5E36mPDuxEvfI9uePI0x6Bw62UPB9524CtP7Xba3Z7Jol5yh4RneF
9fP2+Xpes2WaUSfgU6Cp9KMtqQmdKAtelW0GLOb1/lsW9uuWRRk+EJuloJT993ED
v6TLFuf4DS1B+8c0Yg==
-----END CERTIFICATE-----
 1 s:C = US, O = Google Trust Services, CN = WR2
   i:C = US, O = Google Trust Services LLC, CN = GTS Root R1
-----BEGIN CERTIFICATE-----
MIIFCzCCAvOgAwIBAgIQf/AFoHxM3tEArZlmpRB7mDANBgkqhkiG9w0BAQsFADBH
MQswCQYDVQQGEwJVUzEiMCAGA1UEChMZR29vZ2xlIFRydXN0IFNlcnZpY2VzIExM
QzEUMBIGA1UEAxMLR1RTIFJvb3QgUjEwHhcNMjMxMjEzMDkwMDAwWhcNMjkwMjIw
MTQwMDAwWjA7MQswCQYDVQQGEwJVUzEeMBwGA1UEChMVR29vZ2xlIFRydXN0IFNl
cnZpY2VzMQwwCgYDVQQDEwNXUjIwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEK
AoIBAQCp/5x/RR5wqFOfytnlDd5GVld9vI+aWqxG8YSau5HbyfsvAfuSCQAWXqAc
+MGr+XgvSszYhaLYWTwO0xj7sfUkDSbutltkdnwUxy96zqhMt/TZCPzfhyMlIKji
aeKMTj+xWfpgoh6zySBTGYLKNlNtYE3pAJH8dolcCA8Kwtzxc2vFE24KT3rC8gIc
LrRjg9ox9illMLL7q8Ju26nADrn5Z9TDJVd06wW06Y613ijNzHoU5HEDy01hLmFX
xRmpC5iEGuh5KdmyjS//V2pm4M6rlagplmNwEmceOuHbsCFxl3ye/aoXbv4r+zgX
FNFmp6+atXDMyGOBOozAKql2N87jAgMBAAGjgf4wgfswDgYDVR0PAQH/BAQDAgGG
MB0GA1UdJQQWMBQGCCsGAQUFBwMBBggrBgEFBQcDAjASBgNVHRMBAf8ECDAGAQH/
AgEAMB0GA1UdDgQWBBTeGx7teRXUPjckwyG77DQ5bUKyMDAfBgNVHSMEGDAWgBTk
```

Saving Certificate From openssl

```
echo  \
| openssl s_client -connect www.google.com:443 -servername www.google.com 2>/dev/null  \
| openssl x509 -outform pem  \
> /cygdrive/c/security-course/workshop-https/cert.pem
```

Explanation notes:
« echo | » to force non interactive connect then close
« openssl s_client -connect» to establish TLS handshake 2>/dev/null ignore other logs
« openssl x509… » to convert to PEM file

… Strangely, it is not exactly the same file (format/encoding?) as exported from Chrome ?!
Chrome export in « DER »

Converting between DER <-> PEM ..

openssl x509 -in cert.pem -outform der -out cert.der

openssl x509 -in cert.der -inform der -out cert.pem

Part 2 : Trusted Certificate

Converting *.pem to Java « trustore » JKS format && Running java with Trust Store

keytool -importcert -alias mycert -file cert.pem -keystore truststore.jks

```
java \
   -Djavax.net.ssl.trustStore=truststore.jks \
   -Djavax.net.ssl.trustStorePassword=changeit \
   -jar app.jar
```

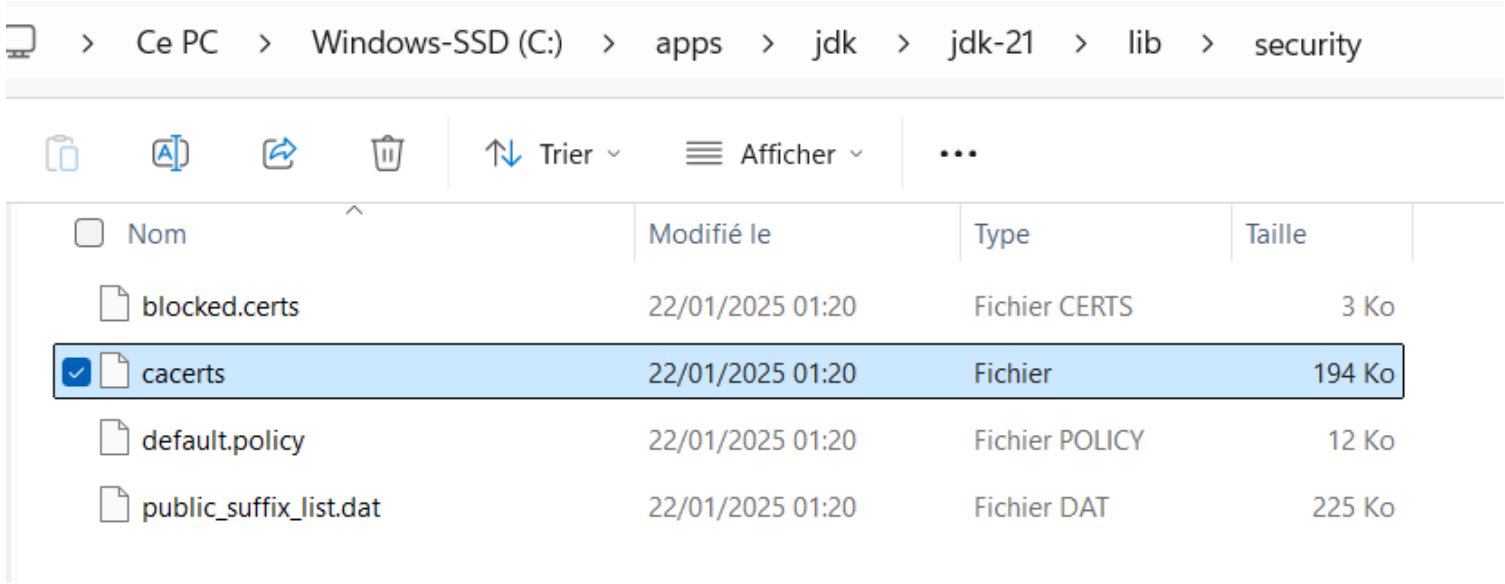
If the http Certificate Chain is not trusted by java => You get error

javax.net.ssl.SSLHandshakeException: sun.security.validator.ValidatorException: PKIX path building failed:
sun.security.provider.certpath.SunCertPathBuilderException:
unable to find valid certification path to requested target

Certificate/Issuers Trusted by default in Java

By default, java trust all certificates from built-in trust store :

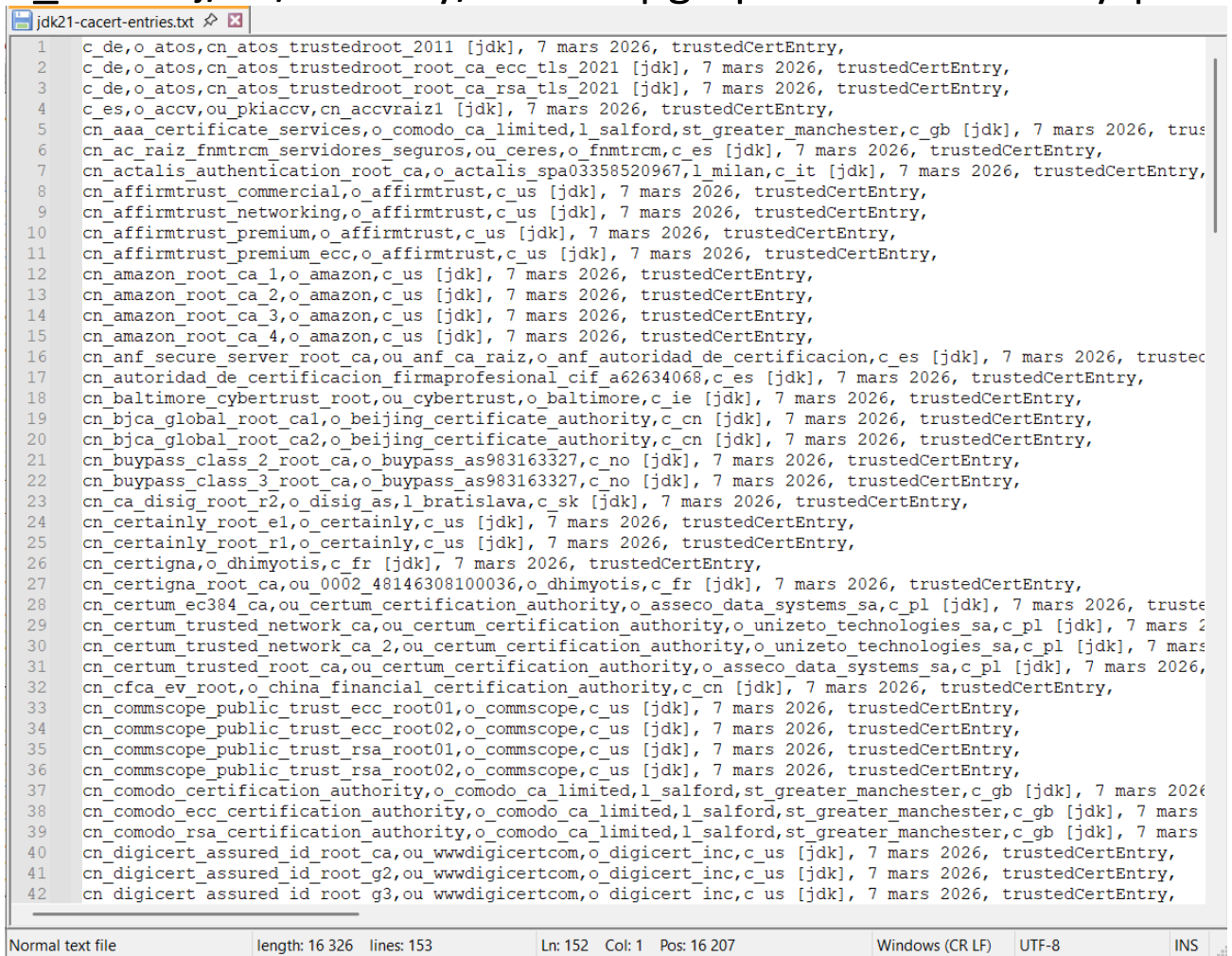
{java.home}/lib/security/cacerts



Listing all Trusted Certs (=> 152 in jdk 21)

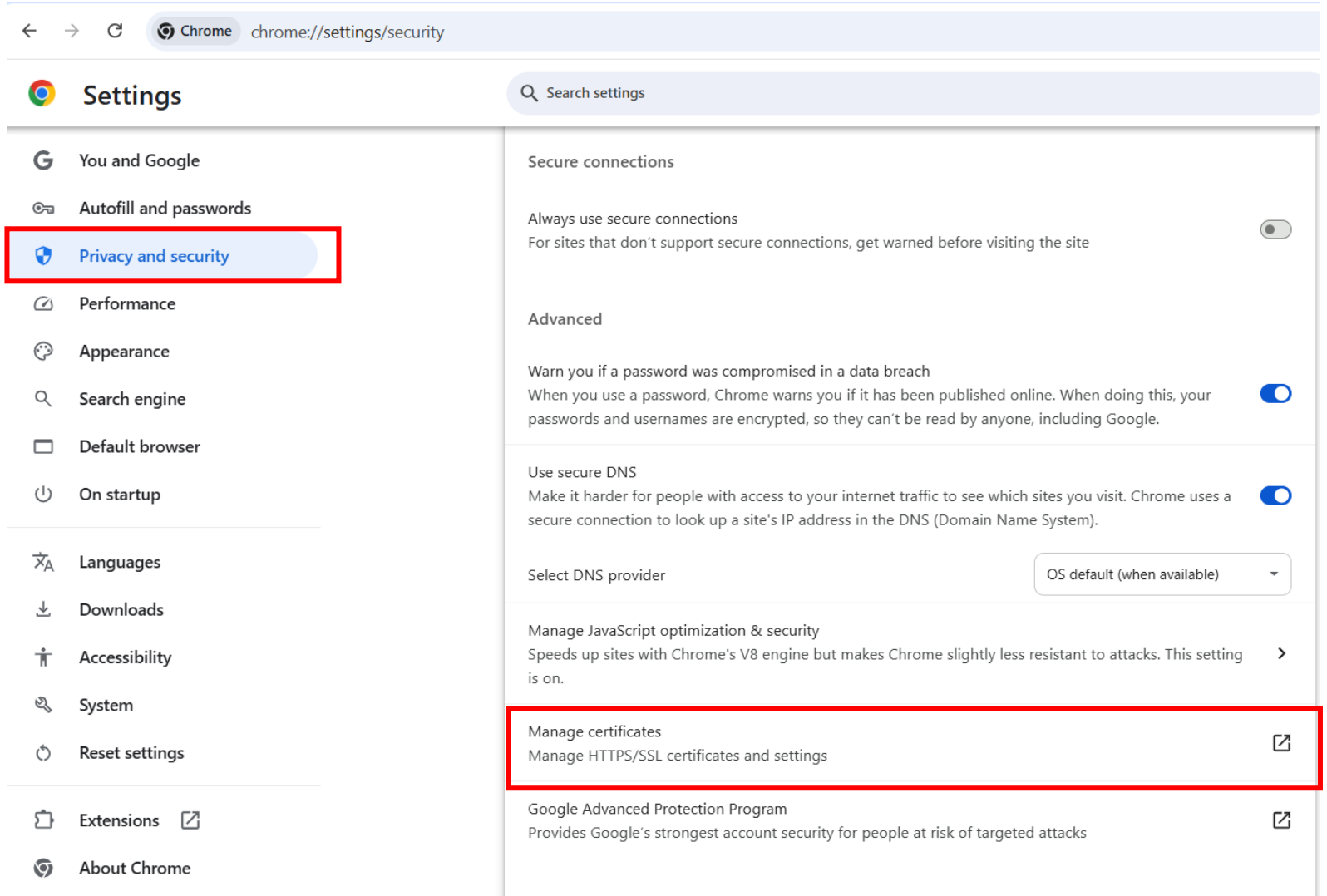
keytool -list -keystore ${JAVA_HOME}/lib/security/cacerts

keytool -list -keystore ${JAVA_HOME}/lib/security/cacerts | grep trustedCertEntry | wc -l

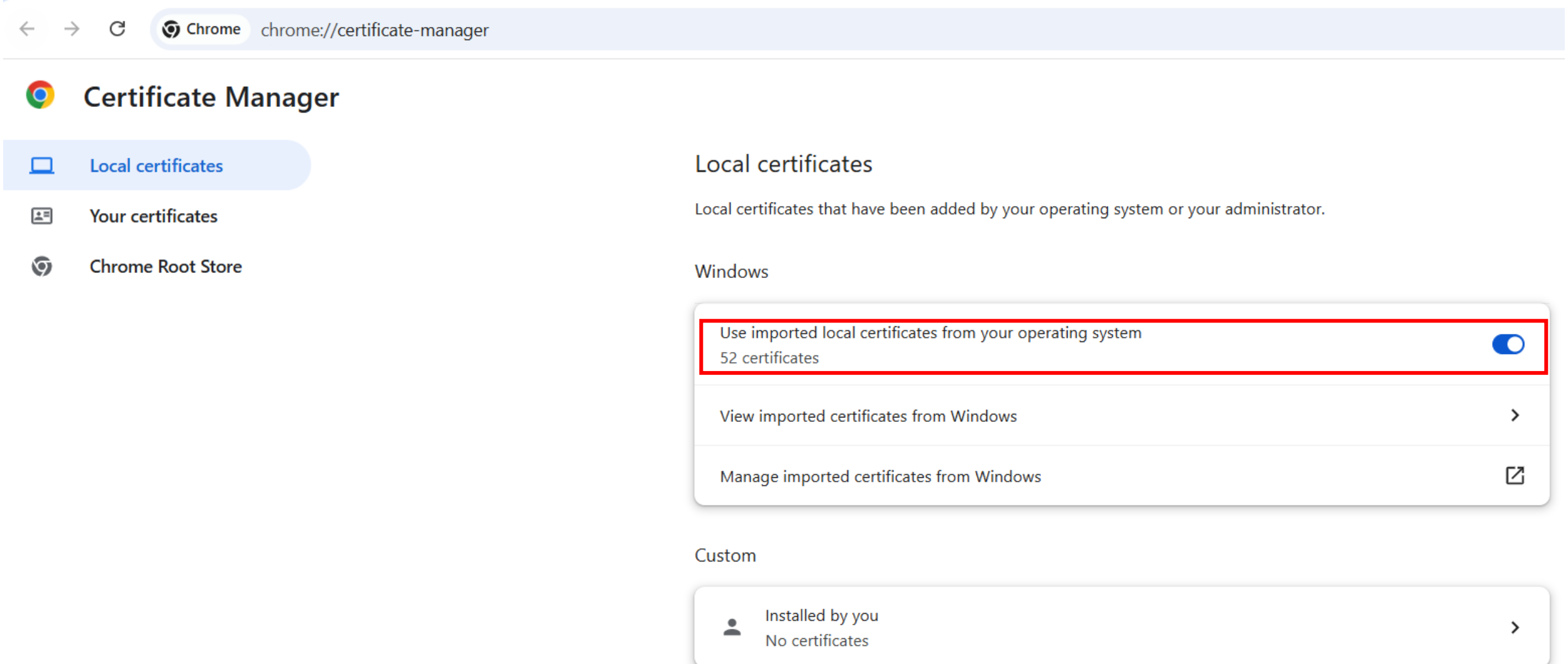


Trusted Certificate by Your Browser

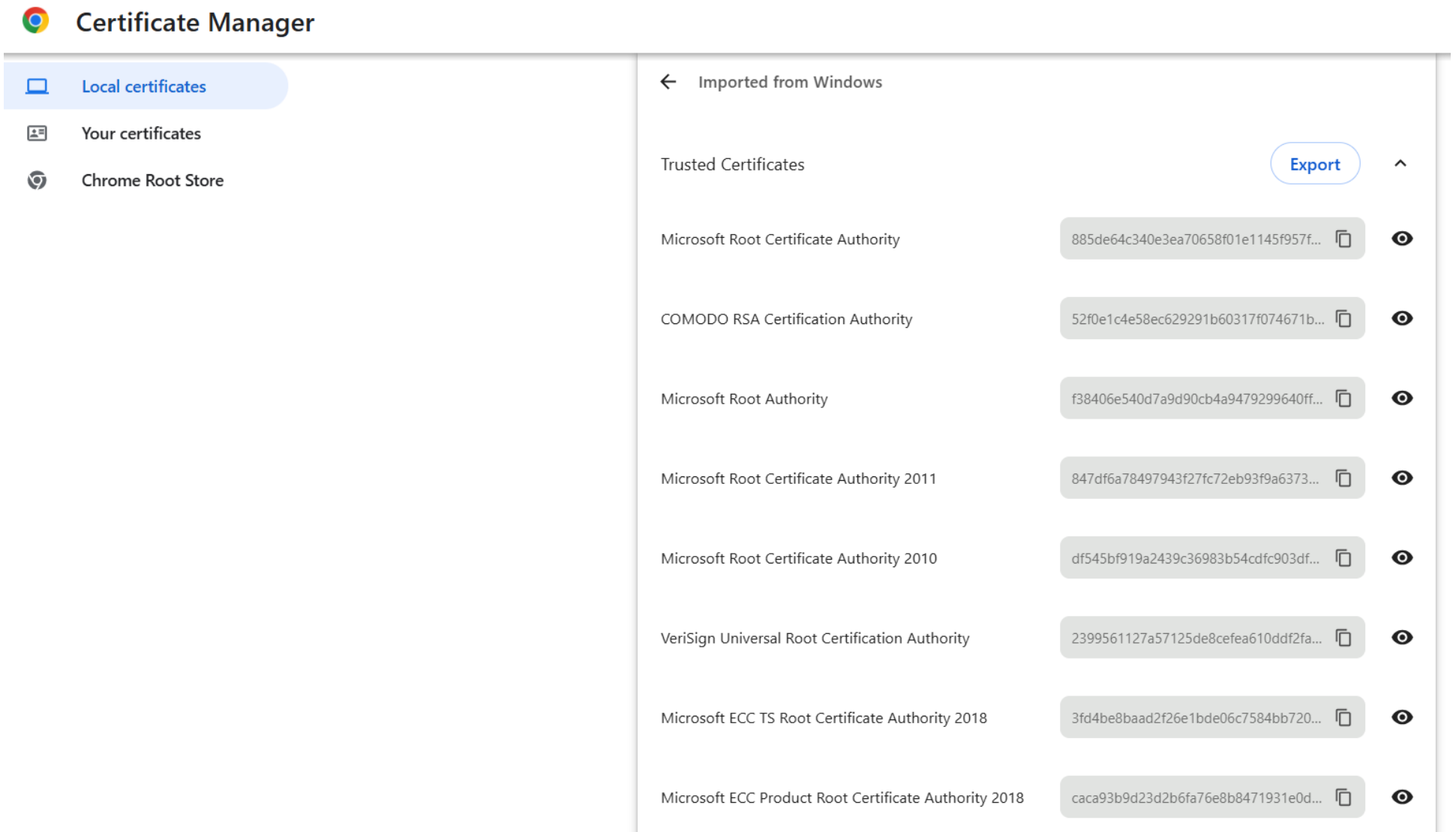
chrome://settings/security



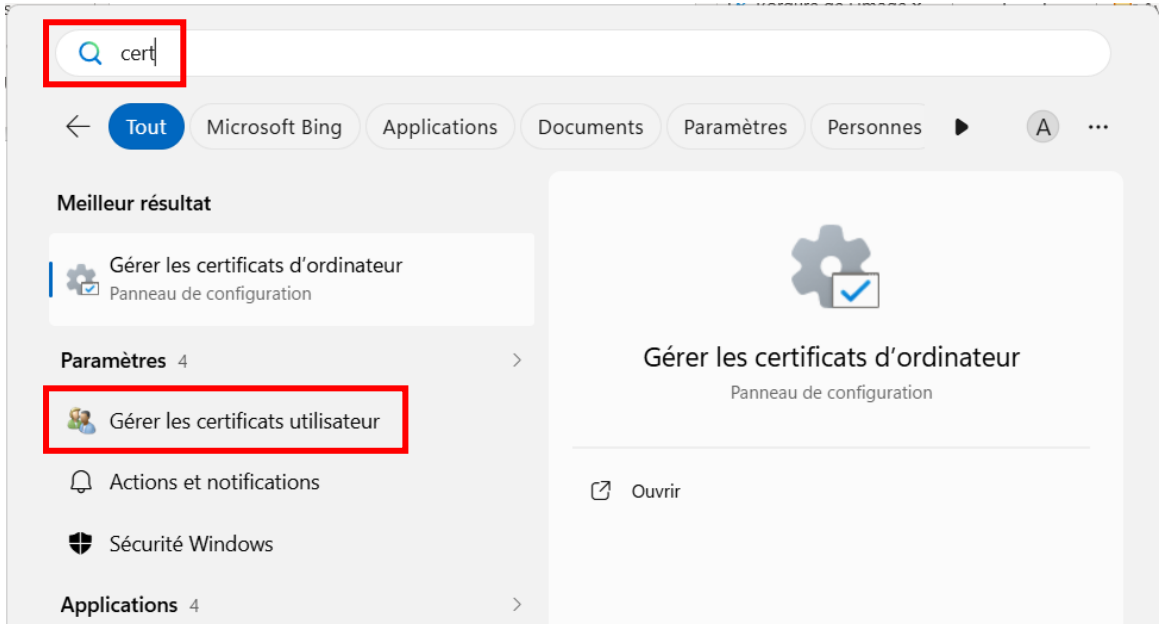
=> rely on your OS (Windows / Mac / …)
52 Certificates

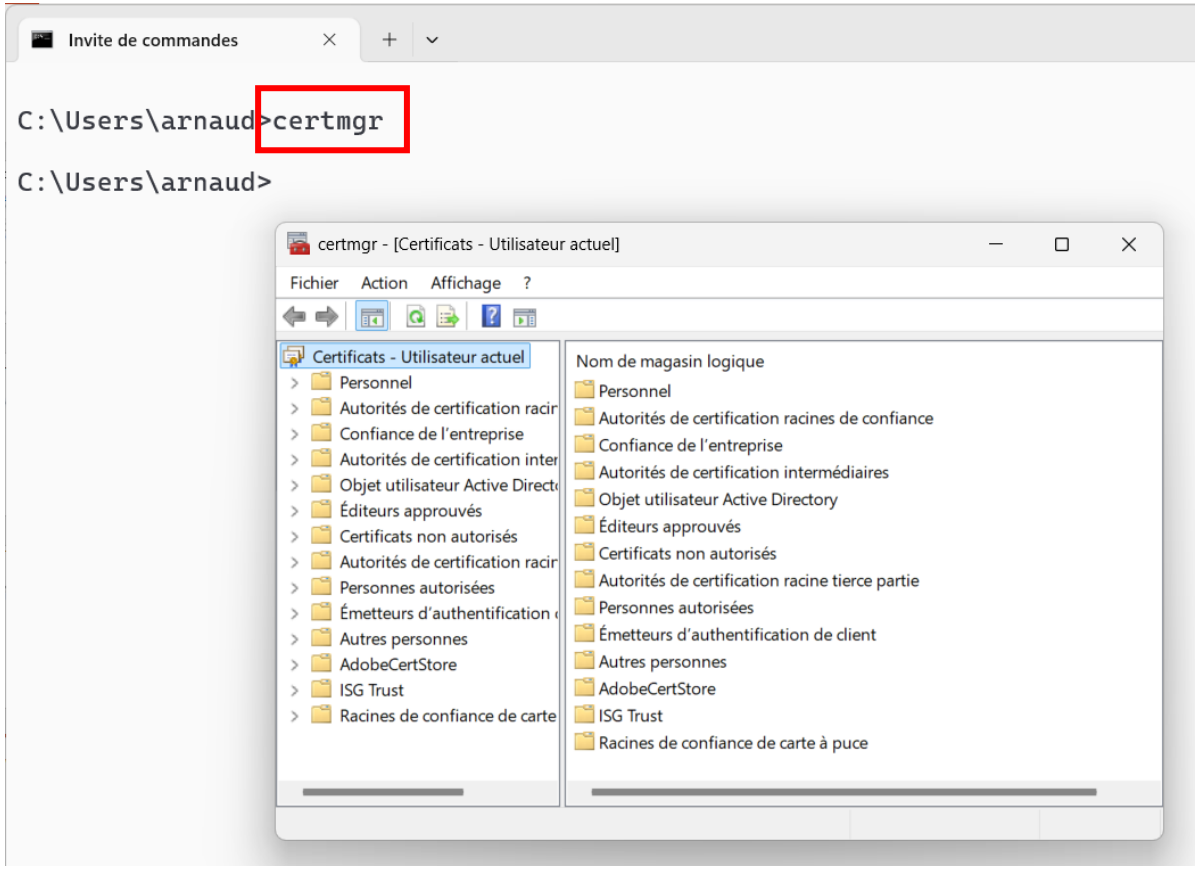


Trusted Certificates …

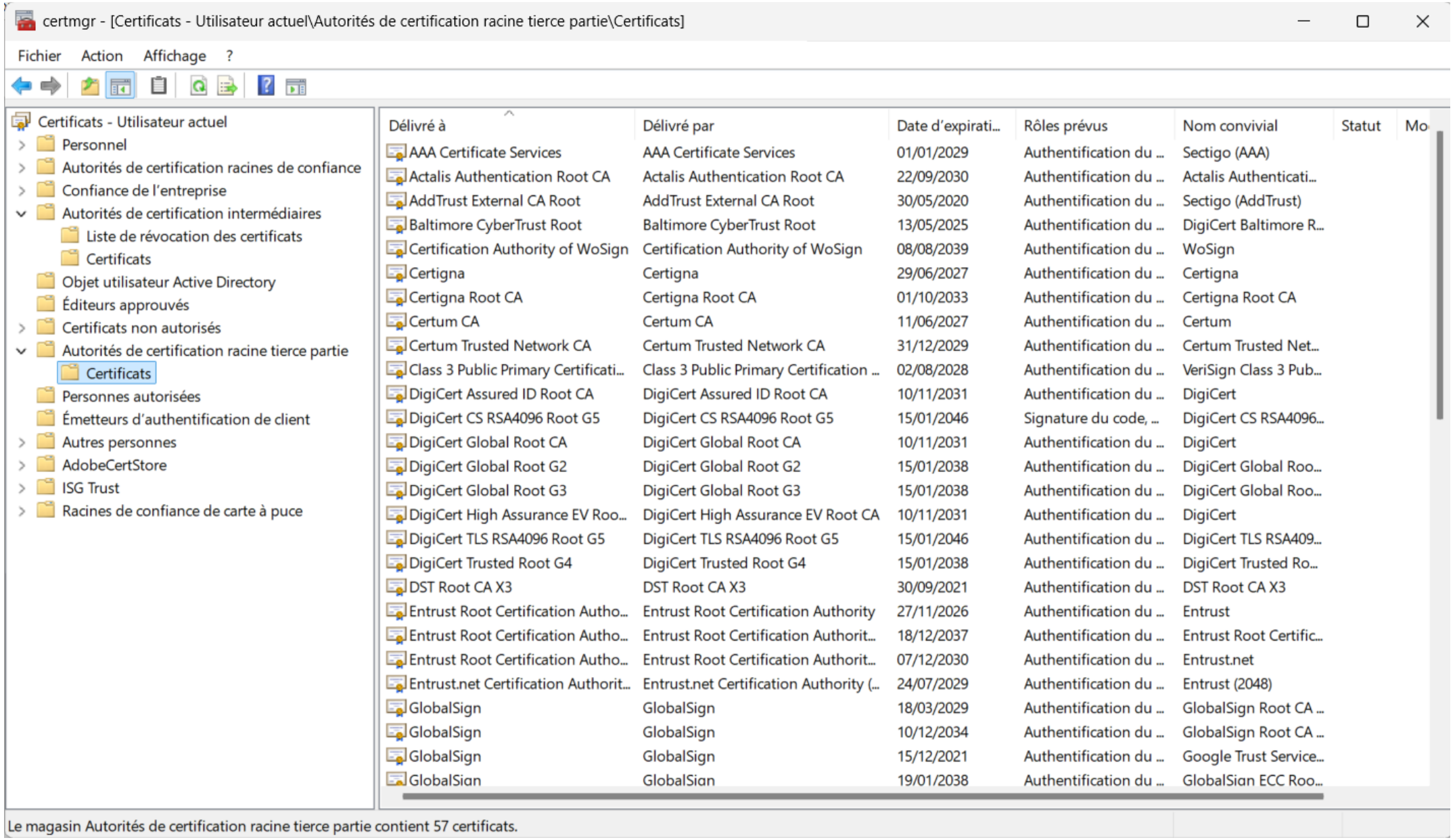


Open Windows Trusted Cert « certmgr »

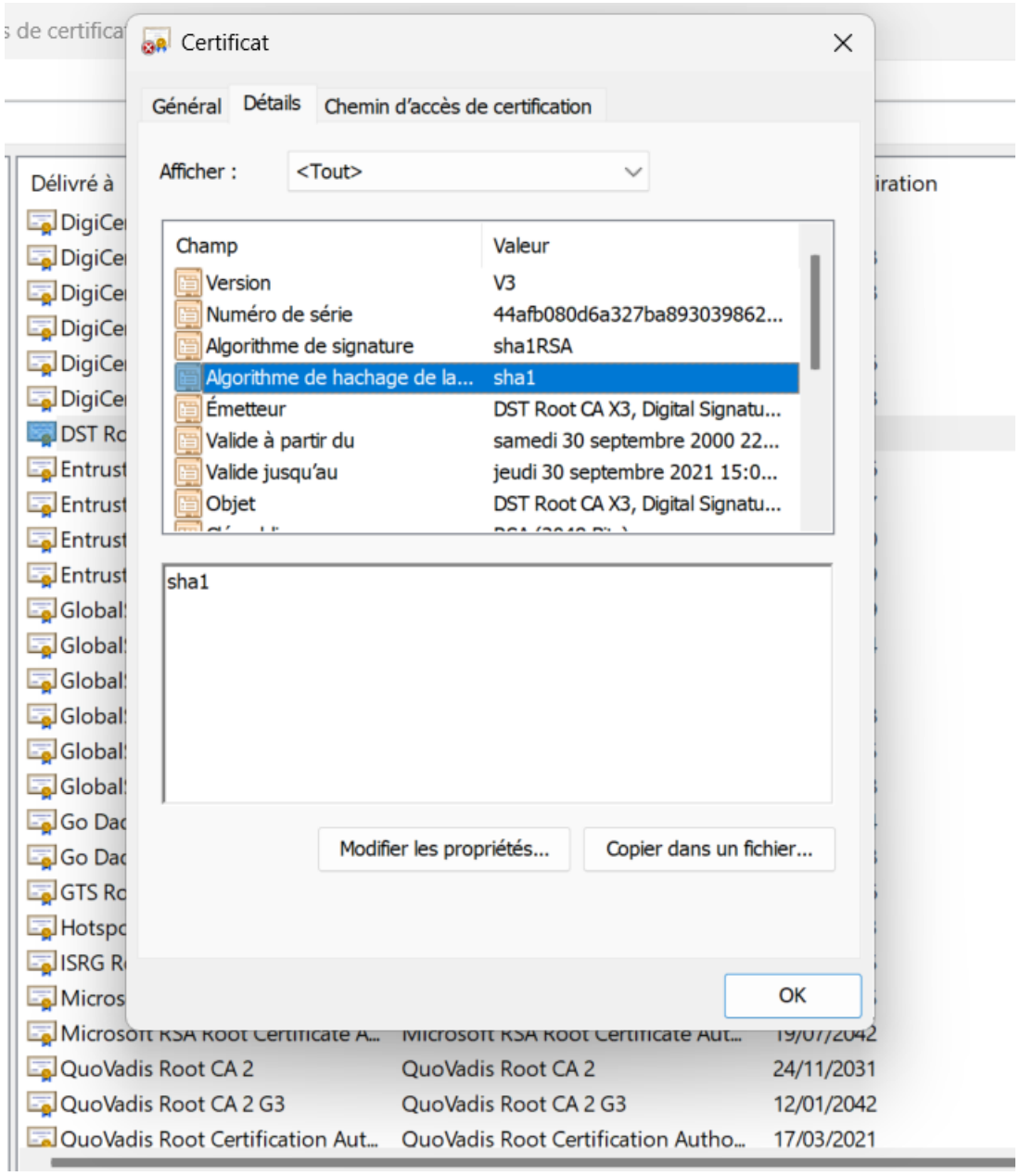




Windows CertMgr (57 certs)



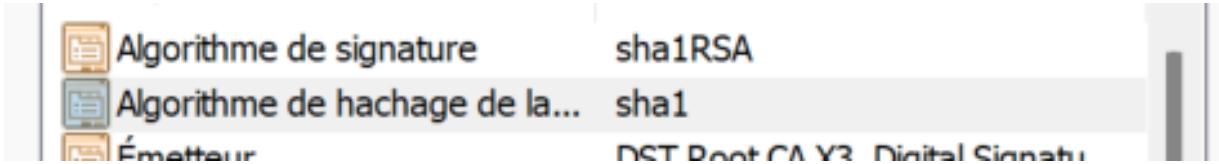
Certificate Details



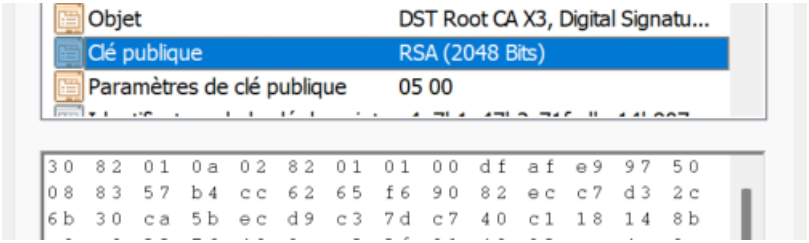
See next…
certificate is stupid ! (expired)

Example of a Deprecated/Unsafe Certificate …

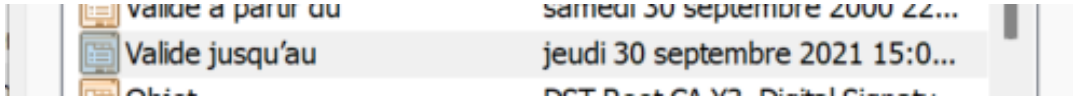
Using SHA-1 ???!!!!!
Recommandation as-of 2026: SHA-256



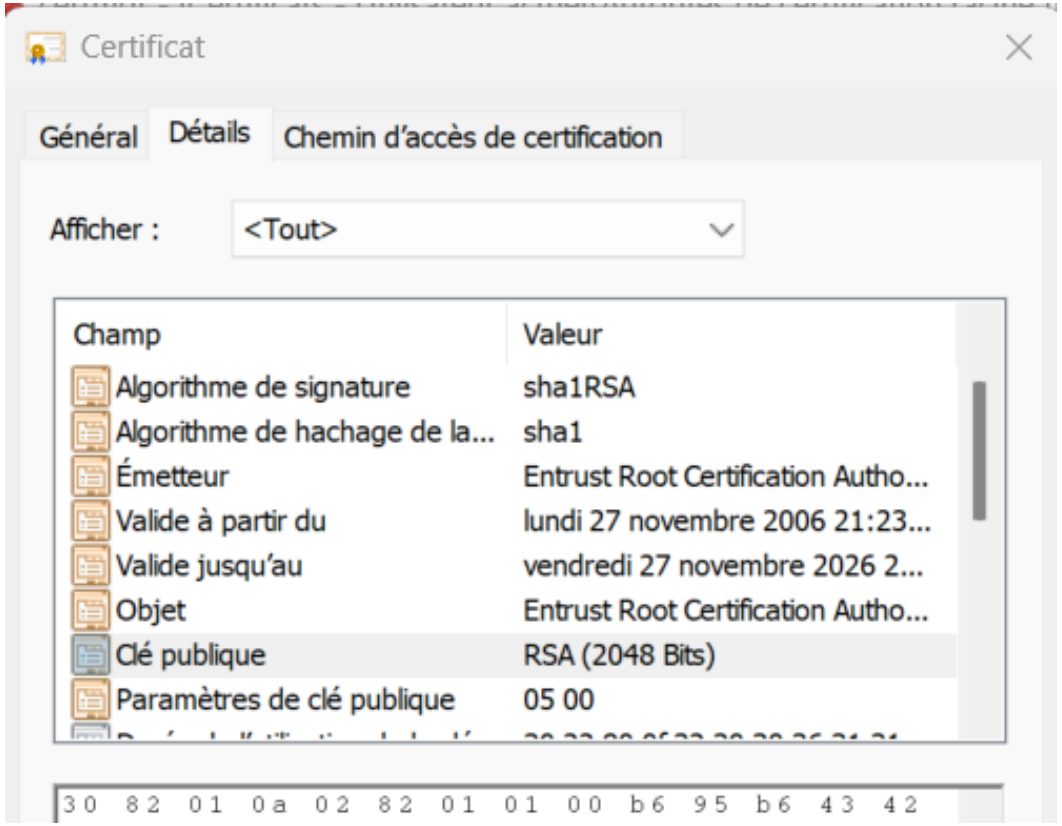
Using only 2028 bits ???!!!!!
Recommandation as-of 2026: >= 4096 bits



Fortunatly… is is expired !! (but 3 years ago, it was already stupid)



Find a valid (as-of 2026), but still stupid: 2048 bits and using SHA-1 ???

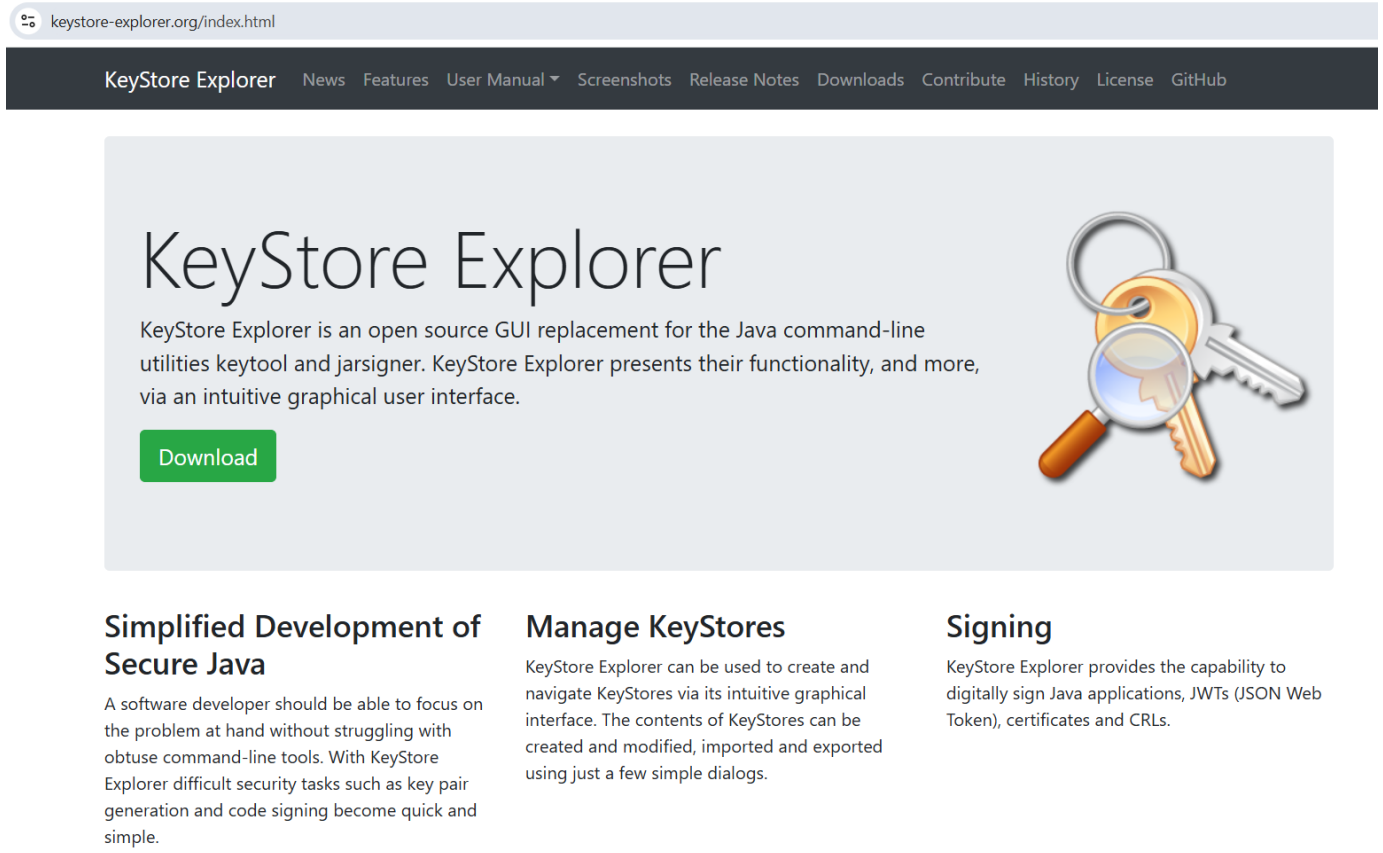


Windows, Chrome, Firefox, and Java reject SHA-1 signed certificates issued after 2016–2017.
So why is it still present?

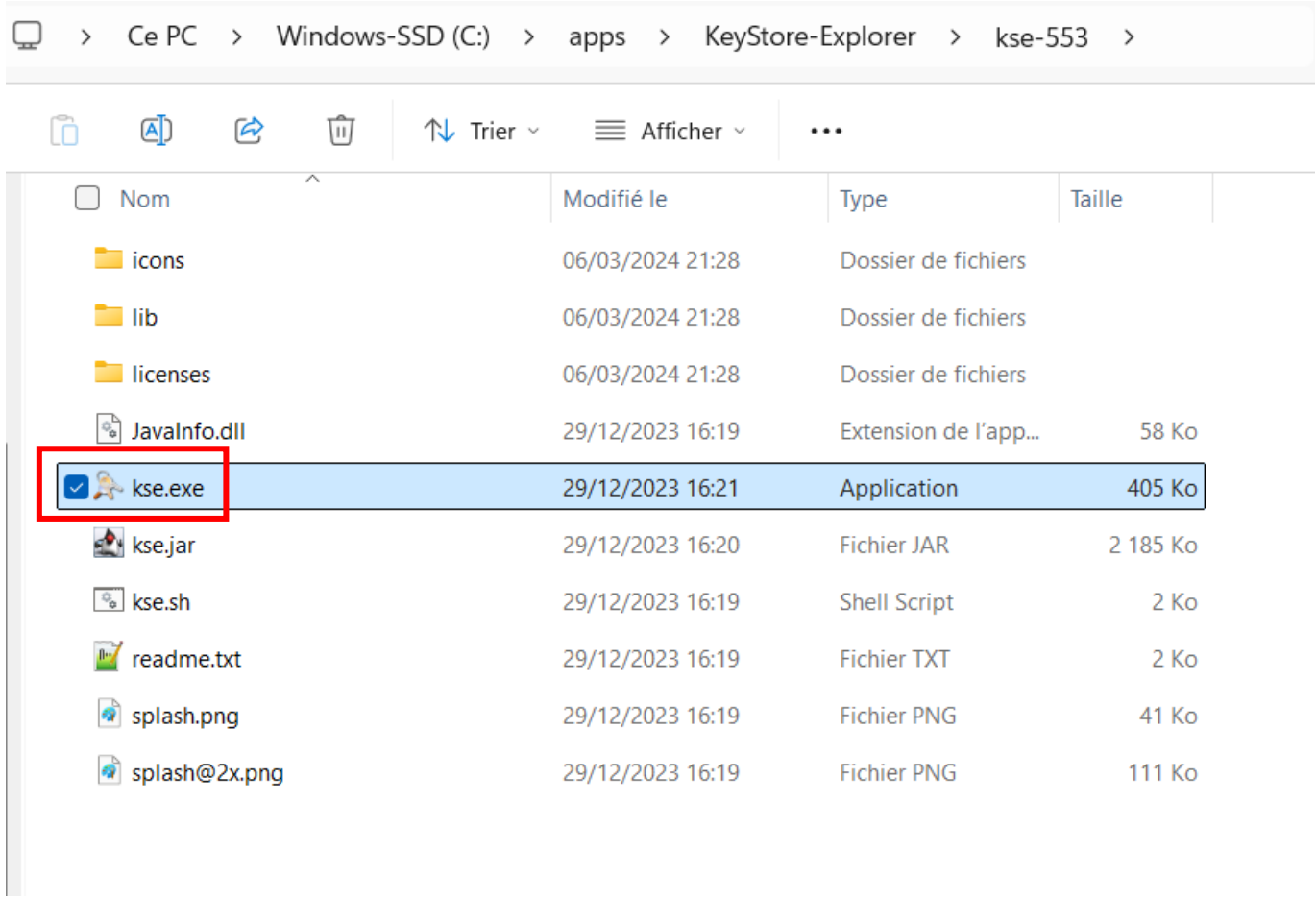
Part 3 … Https Certificate on Server-Side

Generate your Https Self-signed certificate Using KeyStore Explorer UI app

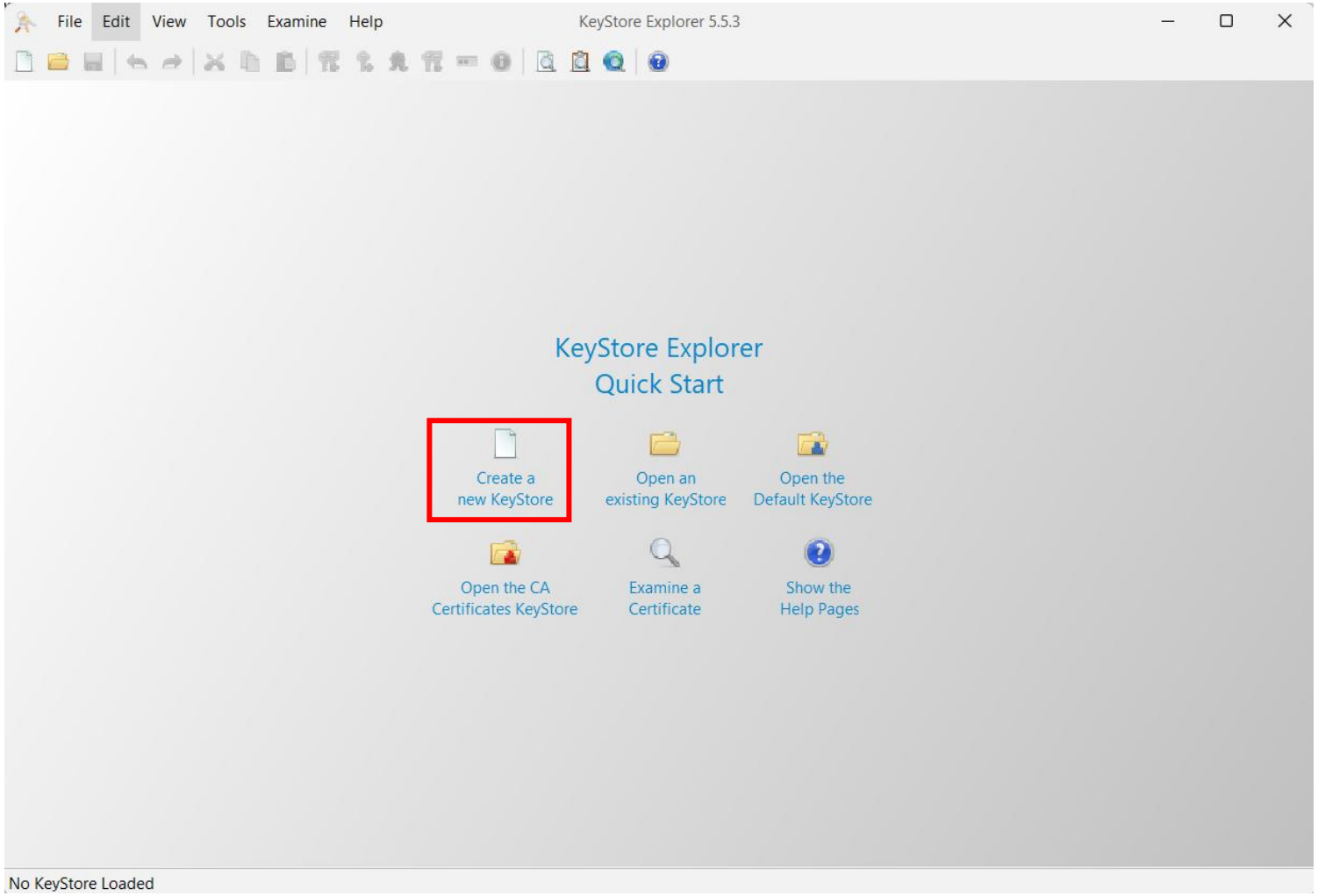
https://keystore-explorer.org/



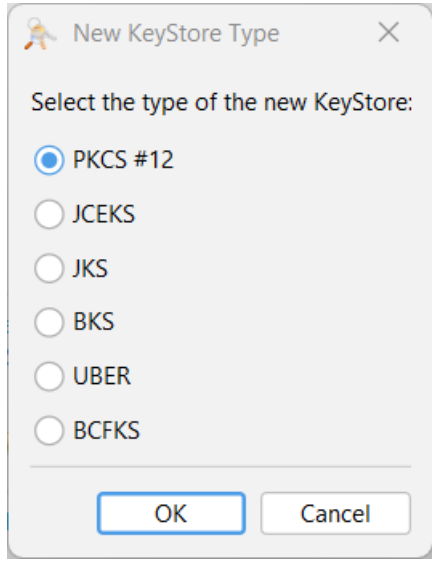
Install, Launch kse.exe



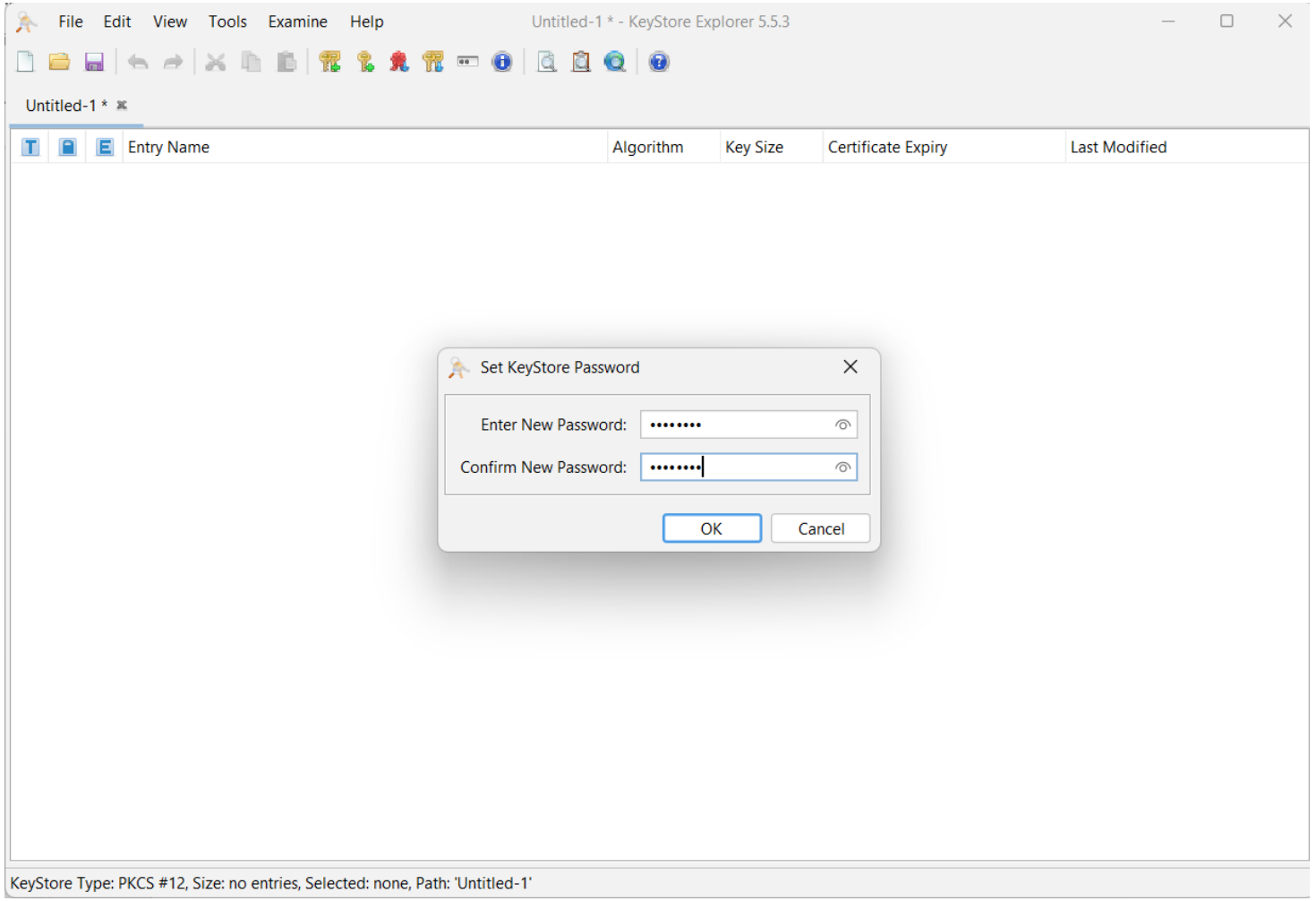
Step 1: Create a New KeyStore

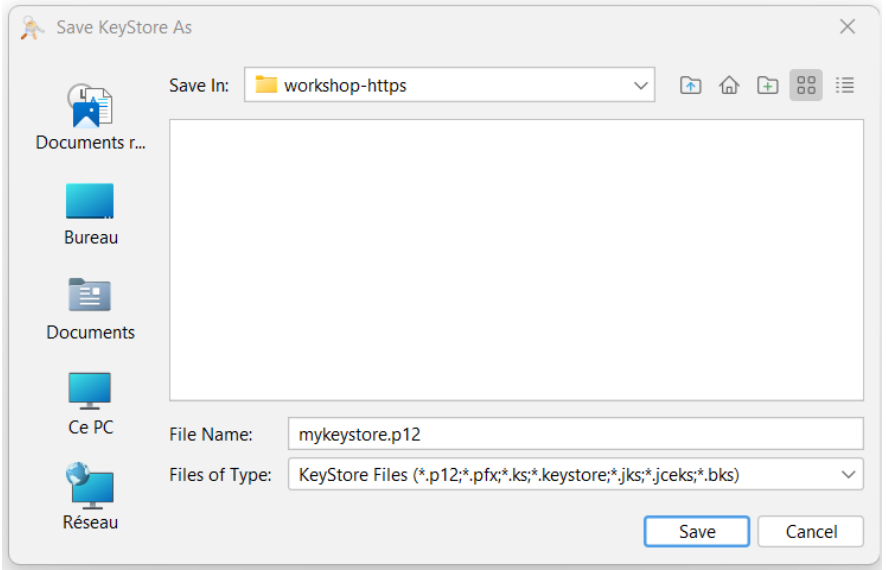


Choose default PKCS#12
(else « JKS » for very old java version <=8)

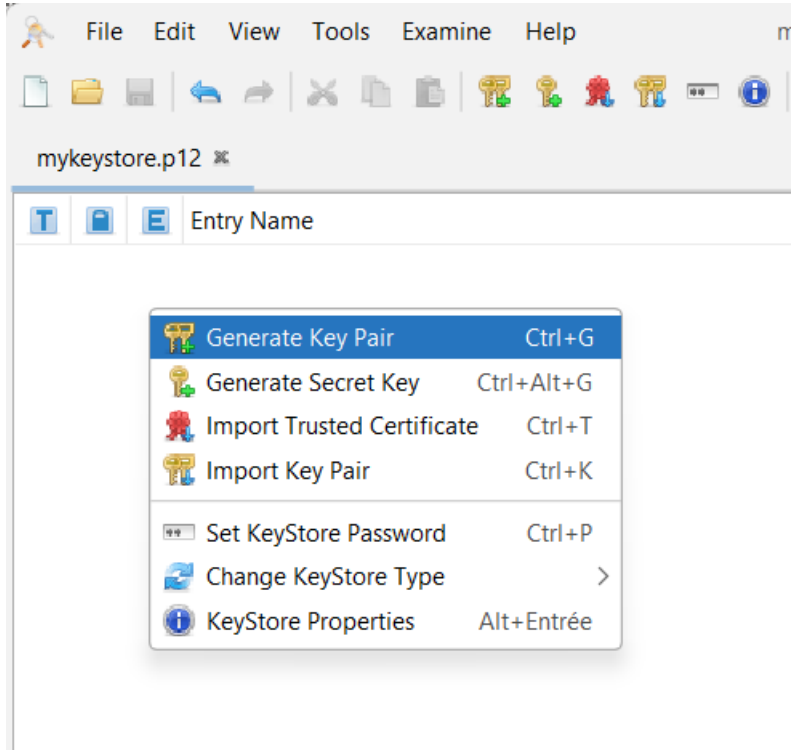


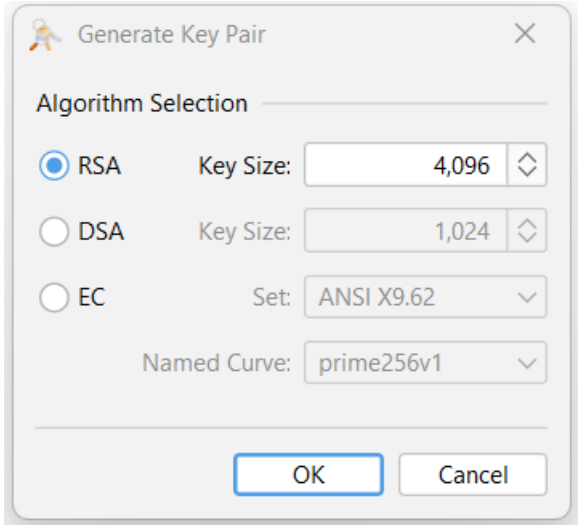
Save your PKCS#12 file (prompt for password, example: changeit »

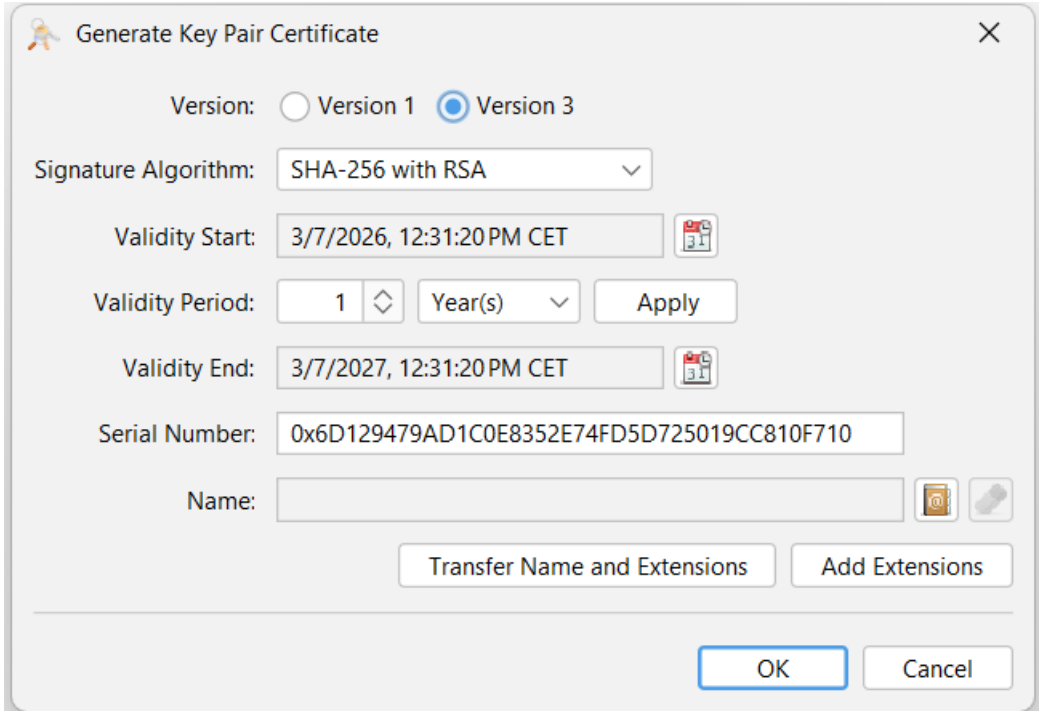




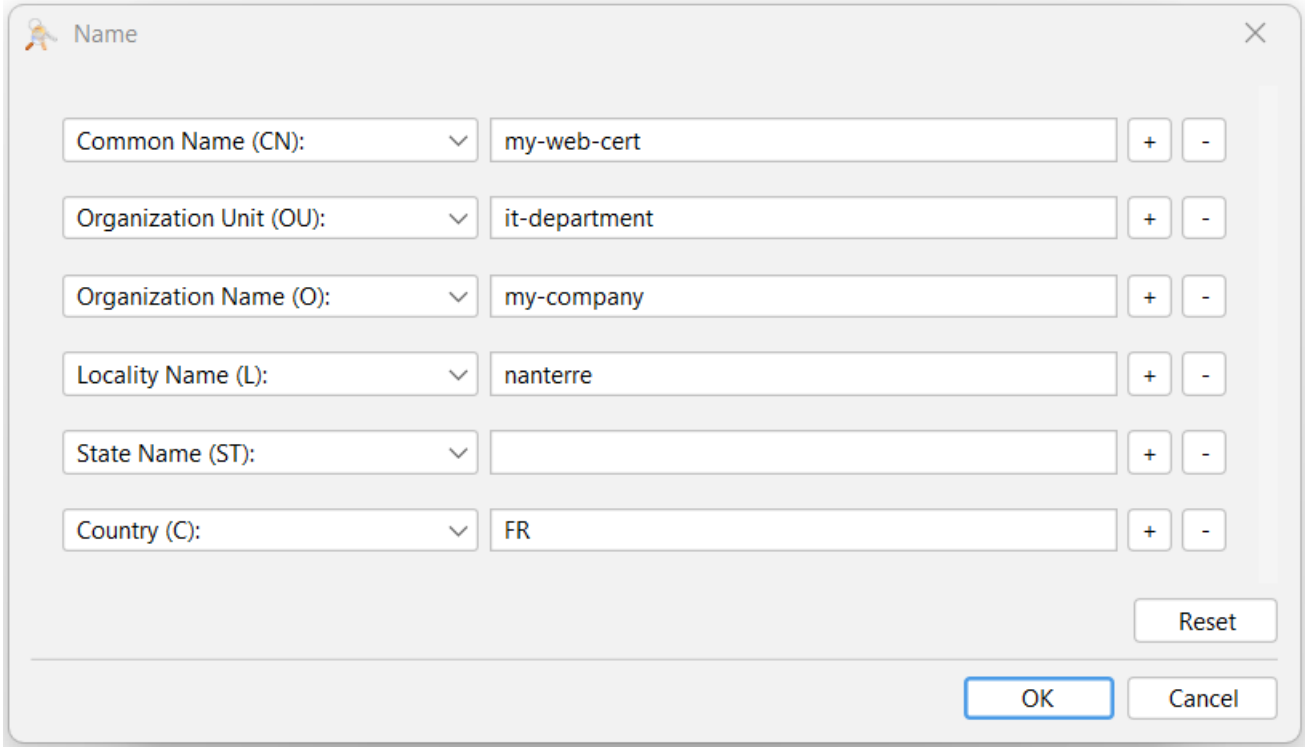
Generate a New KeyPair (private + public part of Certificate)

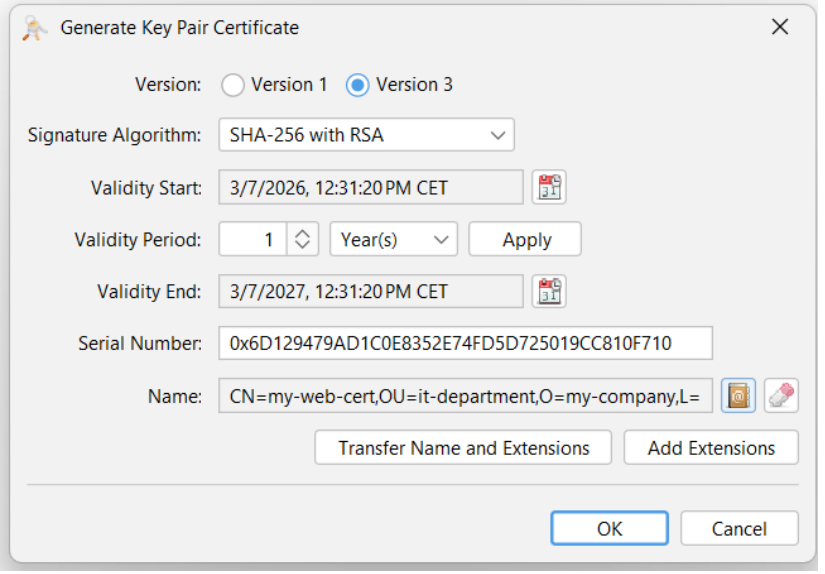




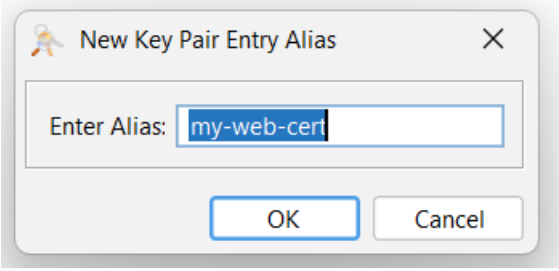


Edit « Names »

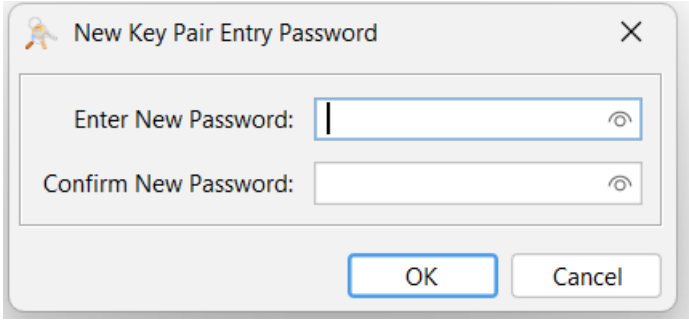

Name
Common Name (CN):
my-web-cert
Organization Unit (OU):
it-department
Organization Name (O):
my-company
Locality Name (L):
nanterre
State Name (ST):
Country (C):
FR
Reset
OK
Cancel


Generate Key Pair Certificate
Version:
Version 1
Version 3
Signature Algorithm:
SHA-256 with RSA
Validity Start:
3/7/2026, 12:31:20 PM CET
Validity Period:
1
Year(s)
Apply
Validity End:
3/7/2027, 12:31:20 PM CET
Serial Number:
0x6D129479AD1C0E8352E74FD5D725019CC810F710
Name:
CN=my-web-cert,OU=it-department,O=my-company,L=
Transfer Name and Extensions
Add Extensions
OK
Cancel

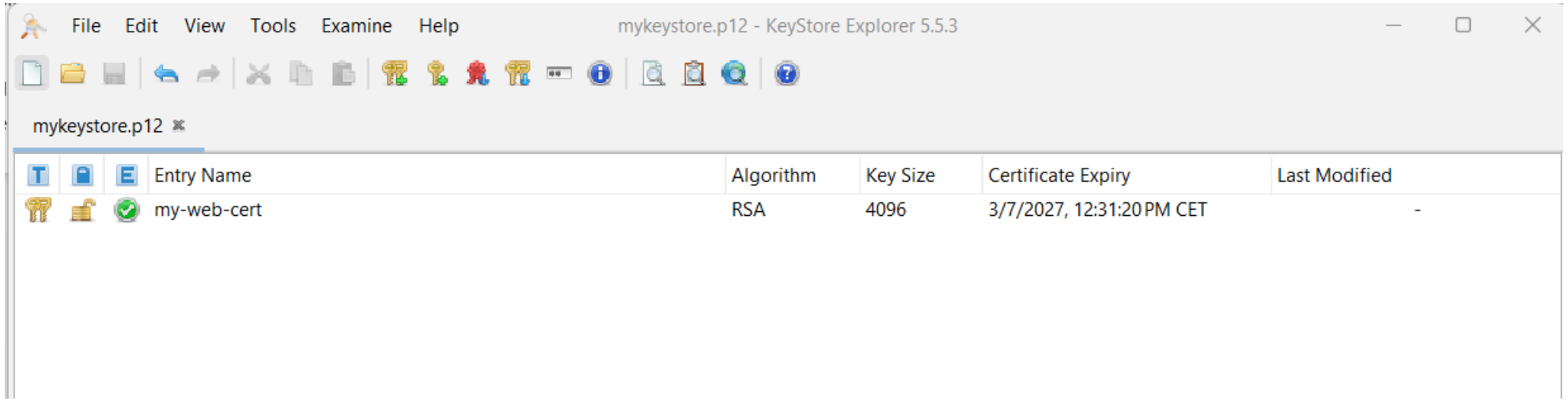
Enter « alias »
(name of KeyPair entry in KeyStore)



Optional… Enter a passphrase for this Entry



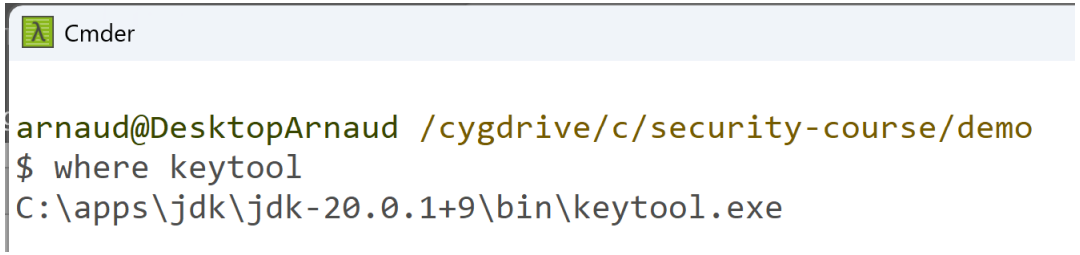
KeyStore now contains a RSA KeyPair Entry



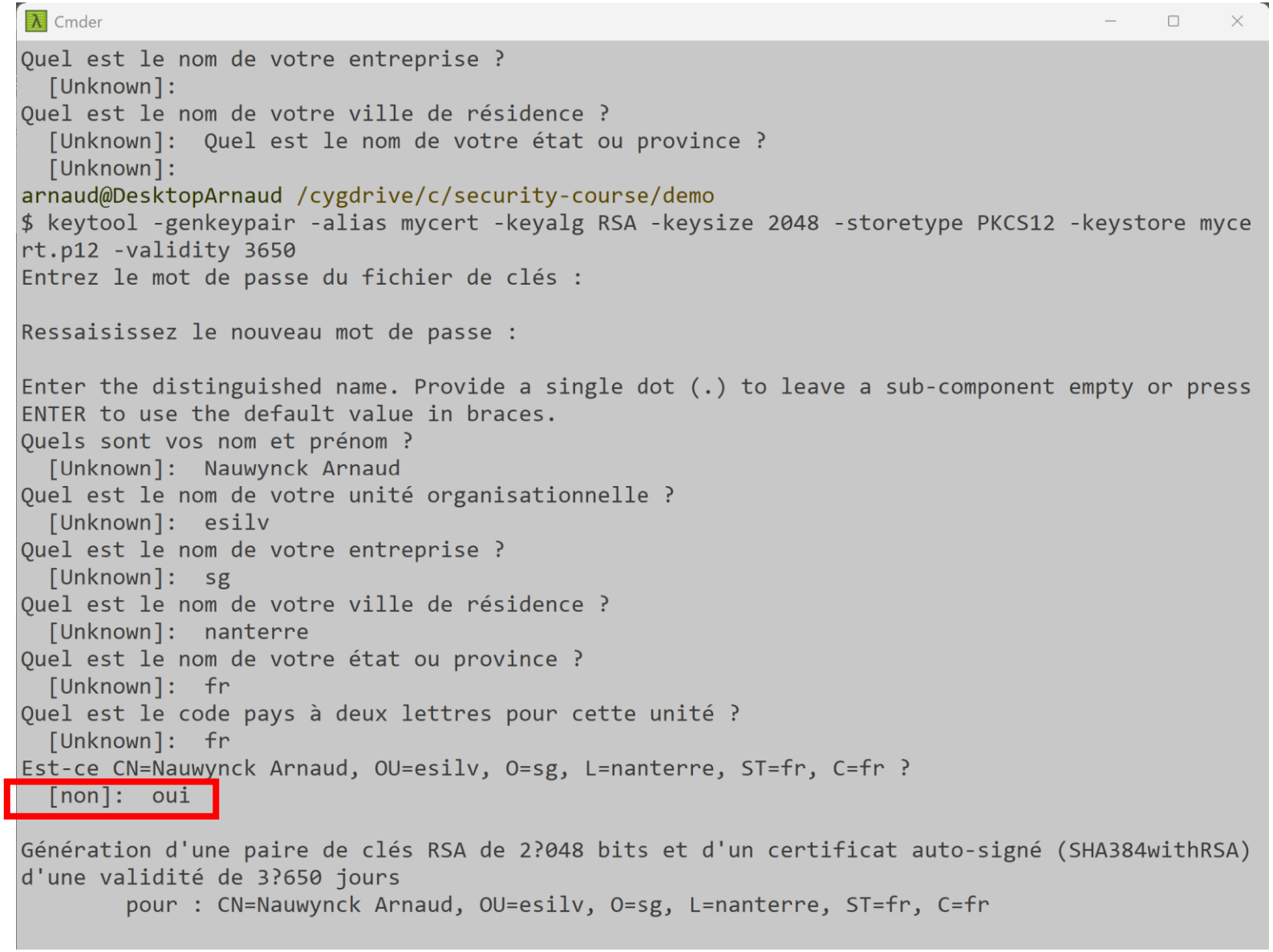
Alternative:
Generate self-signed certificate
From java KeyTool Command Line

keytool -genkeypair -alias mycert -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore mycert.p12 -validity 3650

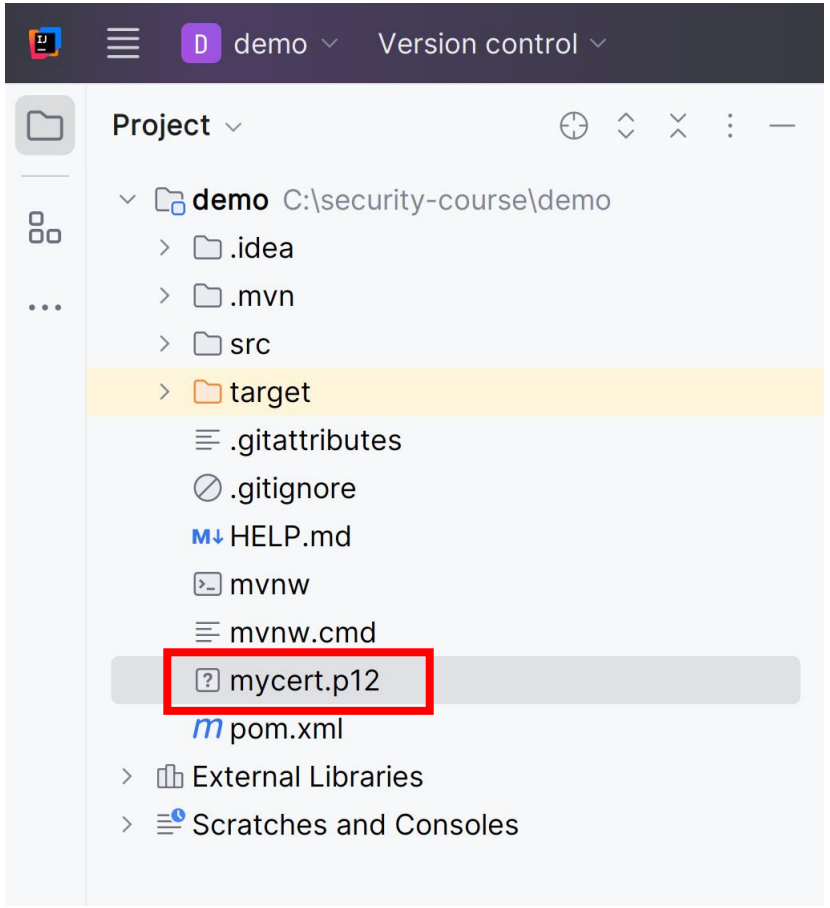
Troubleshooting ?
"keytool" is a tool in your jdk/bin directory
check "where keytool" => it should be found in your "PATH" environment variable
check "echo $PATH"
... to fix, use windows "env" to edit your environement variables, or export PATH="$PATH:$JAVA_HOME/bin"



interactive answer to prompt (any is ok) lastly "Is it CN=.... ? " [no]: type yes !!



Copy file "mycert.p12" in project



Notice… Certificate (self-signed) => Chain

More complex… Not studied here

For curious students :

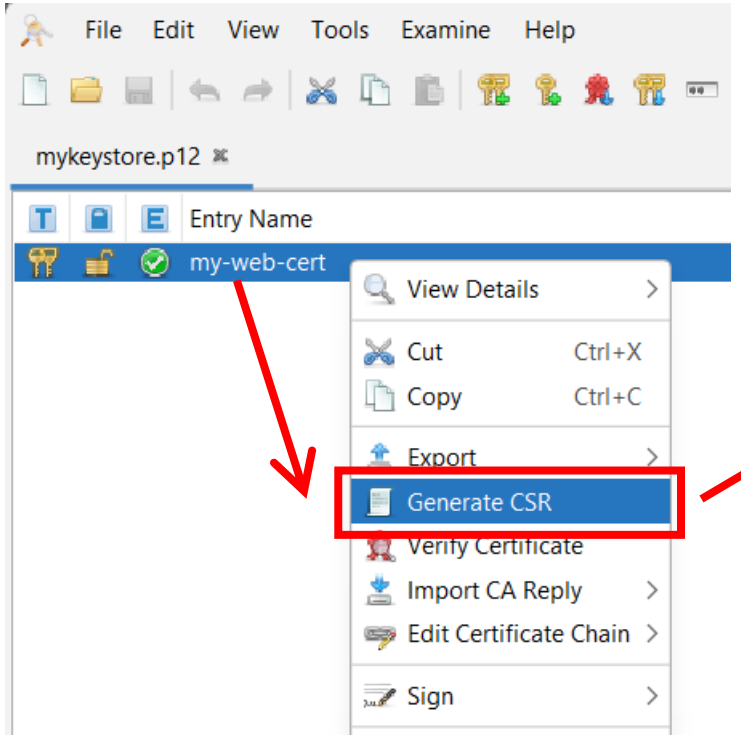
Step 1/ generate a CA = « **Certificate Authority** »
openssl genrsa -out ca.key 4096

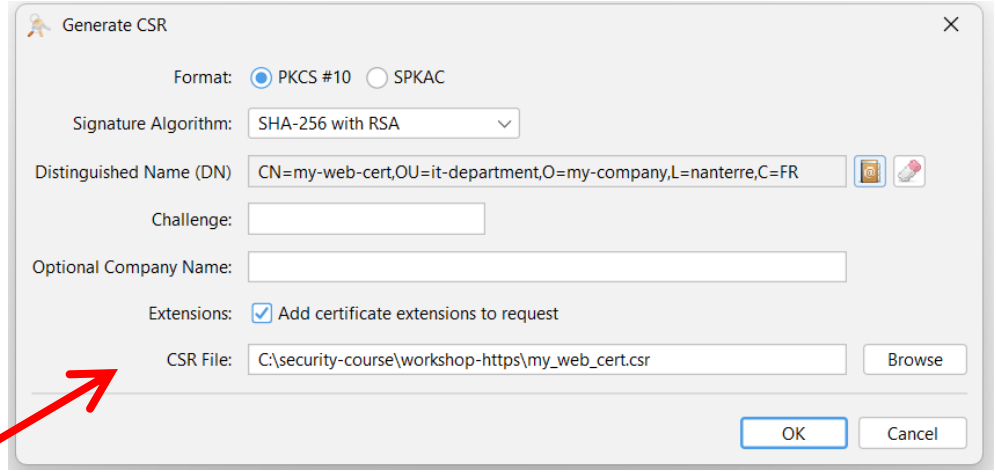
Step 2/ Generate a « CA Certificate » (self-signed … chicken & egg problem)
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt

Step 3/ Generate a CSR = « **Certificate Signing Request** »
openssl req -new -key server.key -out server.csr

Step 4/ **Sign the CSR With the CA, with Alternate Subject Names => CRT**
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256 -extfile san.cnf

Manual Interactive CA + CSR + Sign with KeyStoreExplorer?

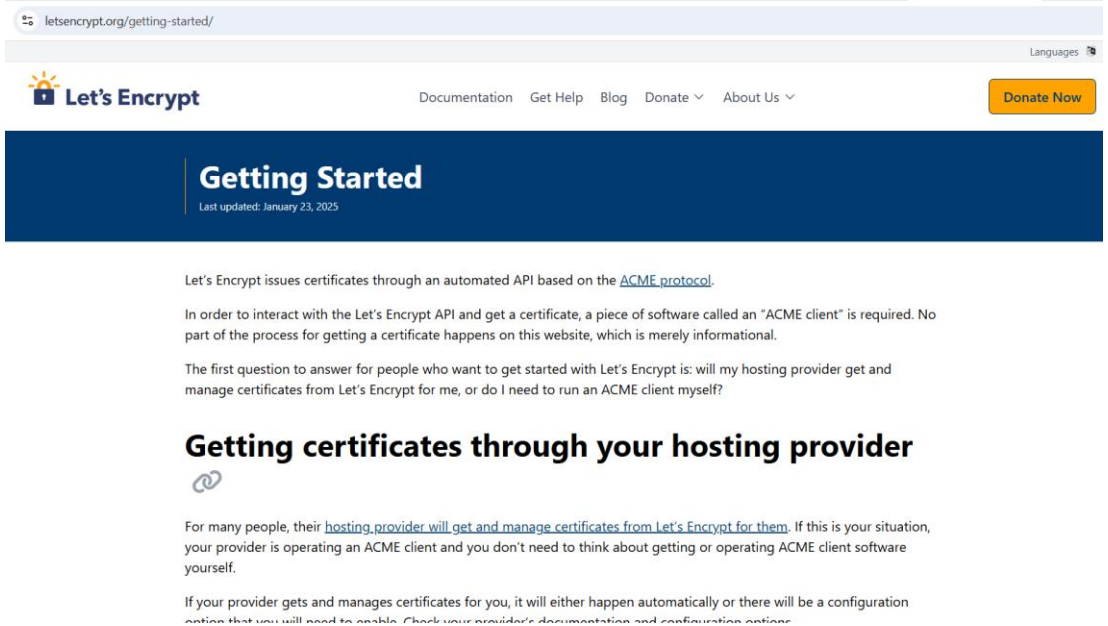




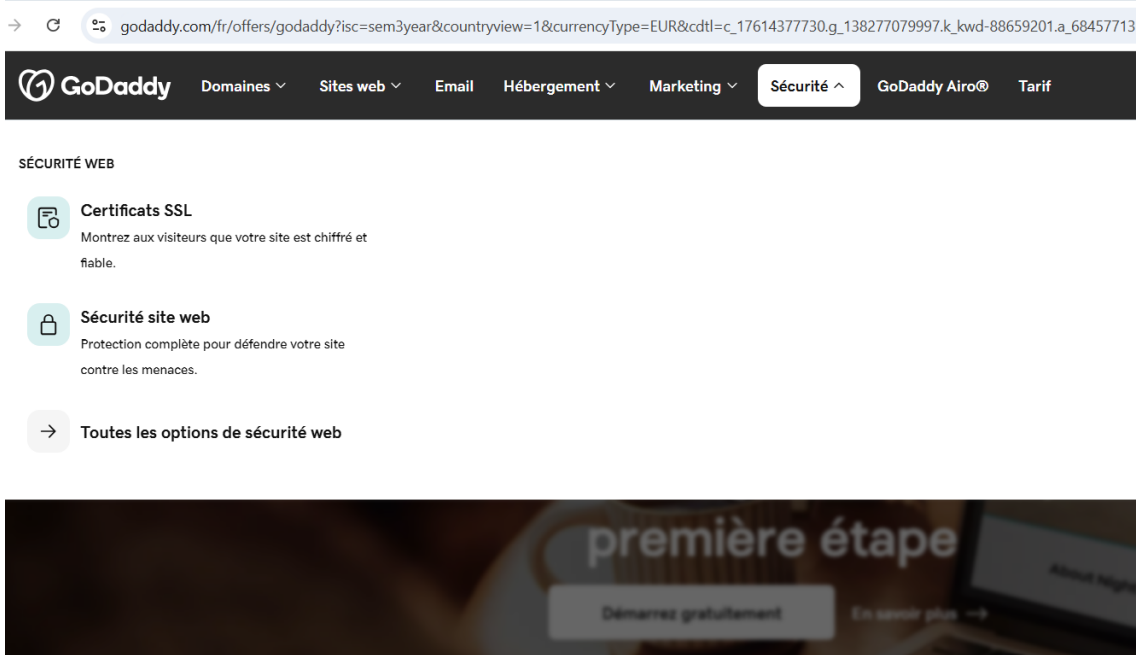


Example of Issuers:
« Let’s Encrypt », « Go Daddy », etc.

https://letsencrypt.org/

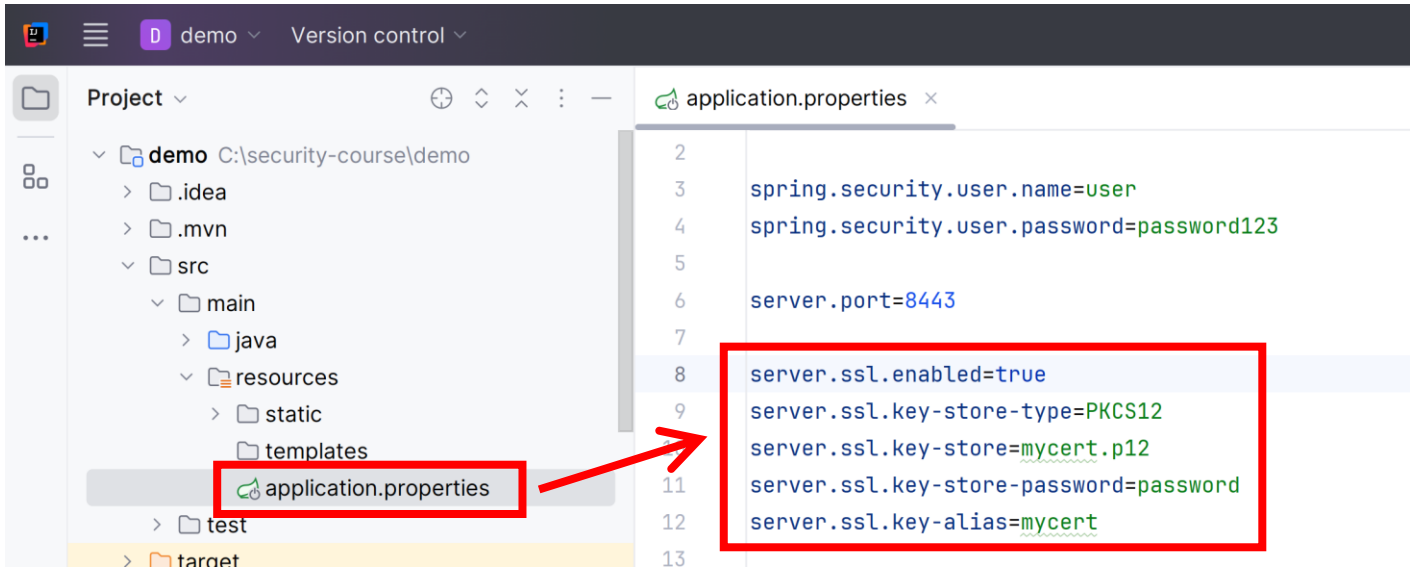


https://www.godaddy.com/



Re-Open your springboot « web » demo project

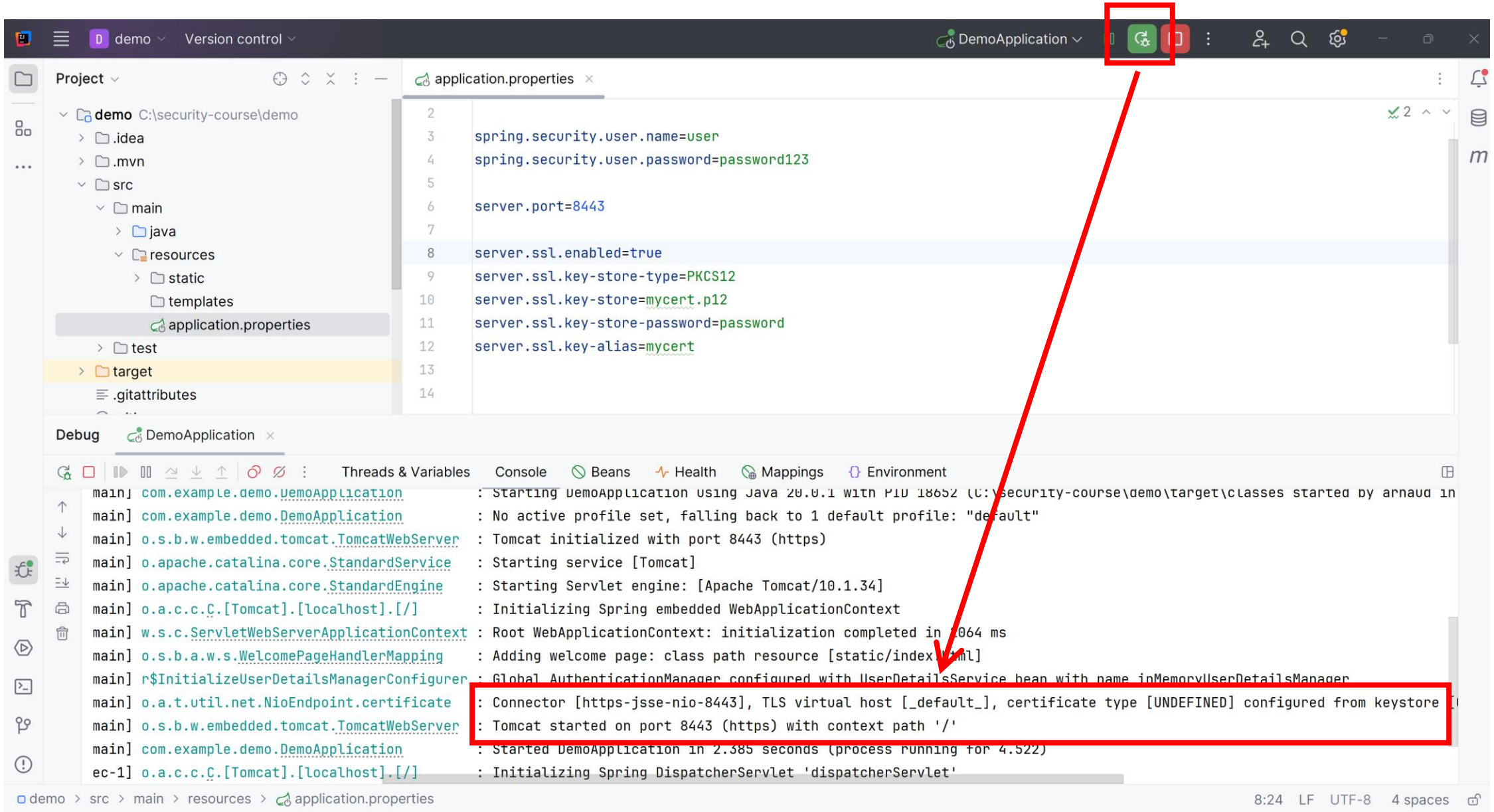
Edit src/main/resources/application.properties to configure SSL



server.port=8443

server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=mycert.p12
server.ssl.key-store-password=password
server.ssl.key-alias=mycert

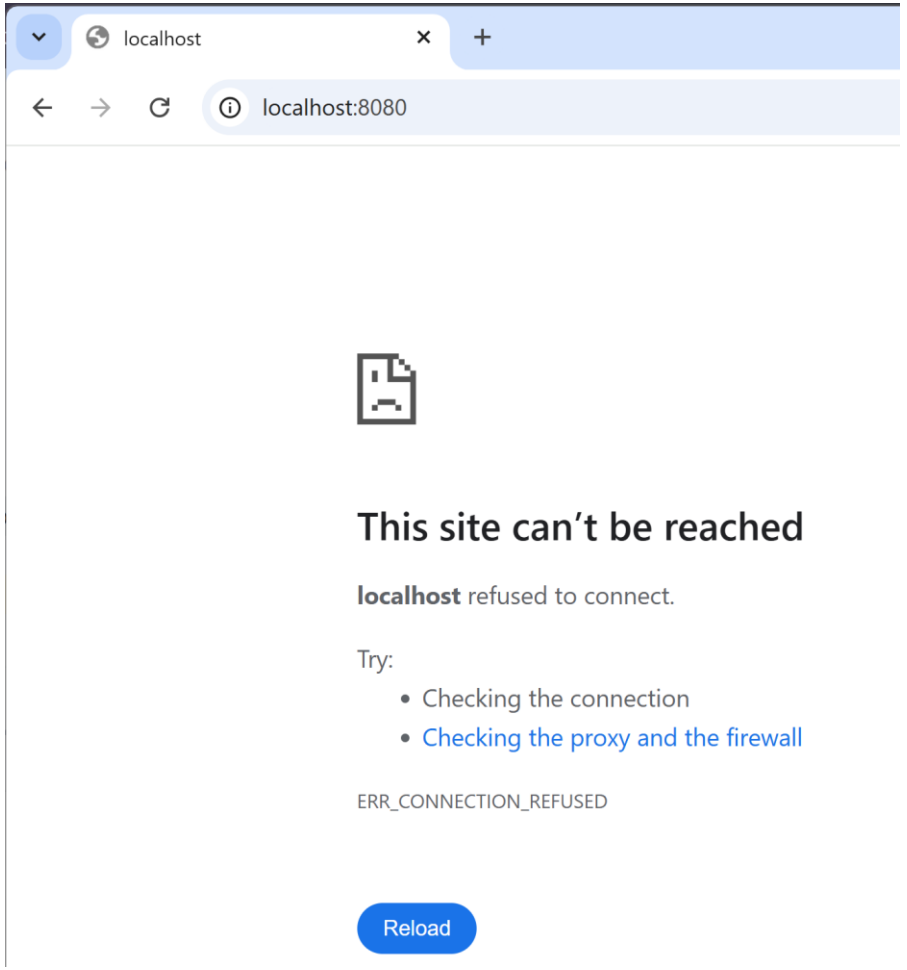
Relaunch server, check new console log



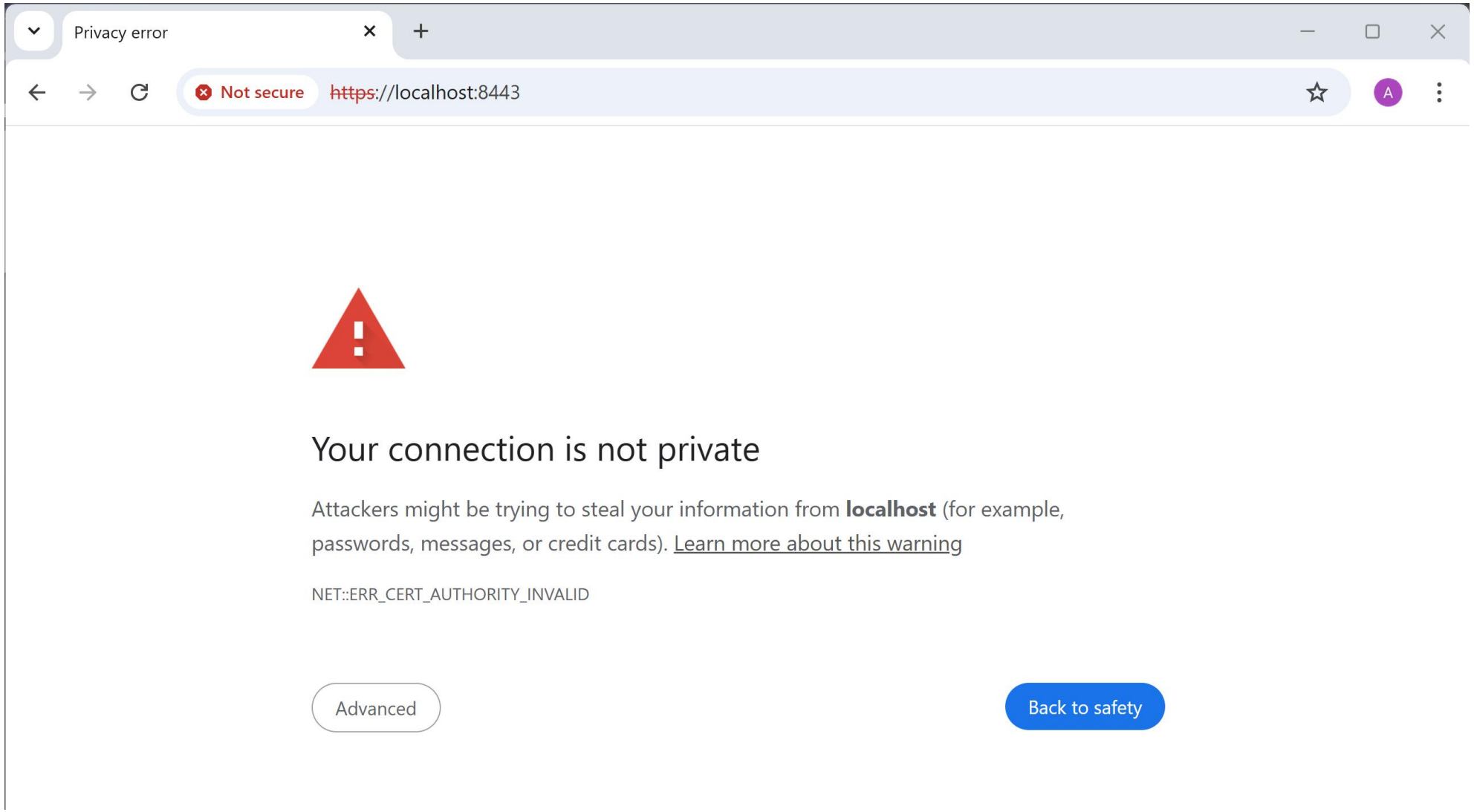
port changed 8080 (default for http) -> 8443 (default for https)

on previous port 8080
=> "ERROR Connection refused"

OK, Normal !!



https://localhost:8443 (self-signed certificate) CERT_AUTHORITY_INVALID ... OK



Why ?
... because of self-signed certificate
NOT built-in recognized in your Chrome

Alternatives ?

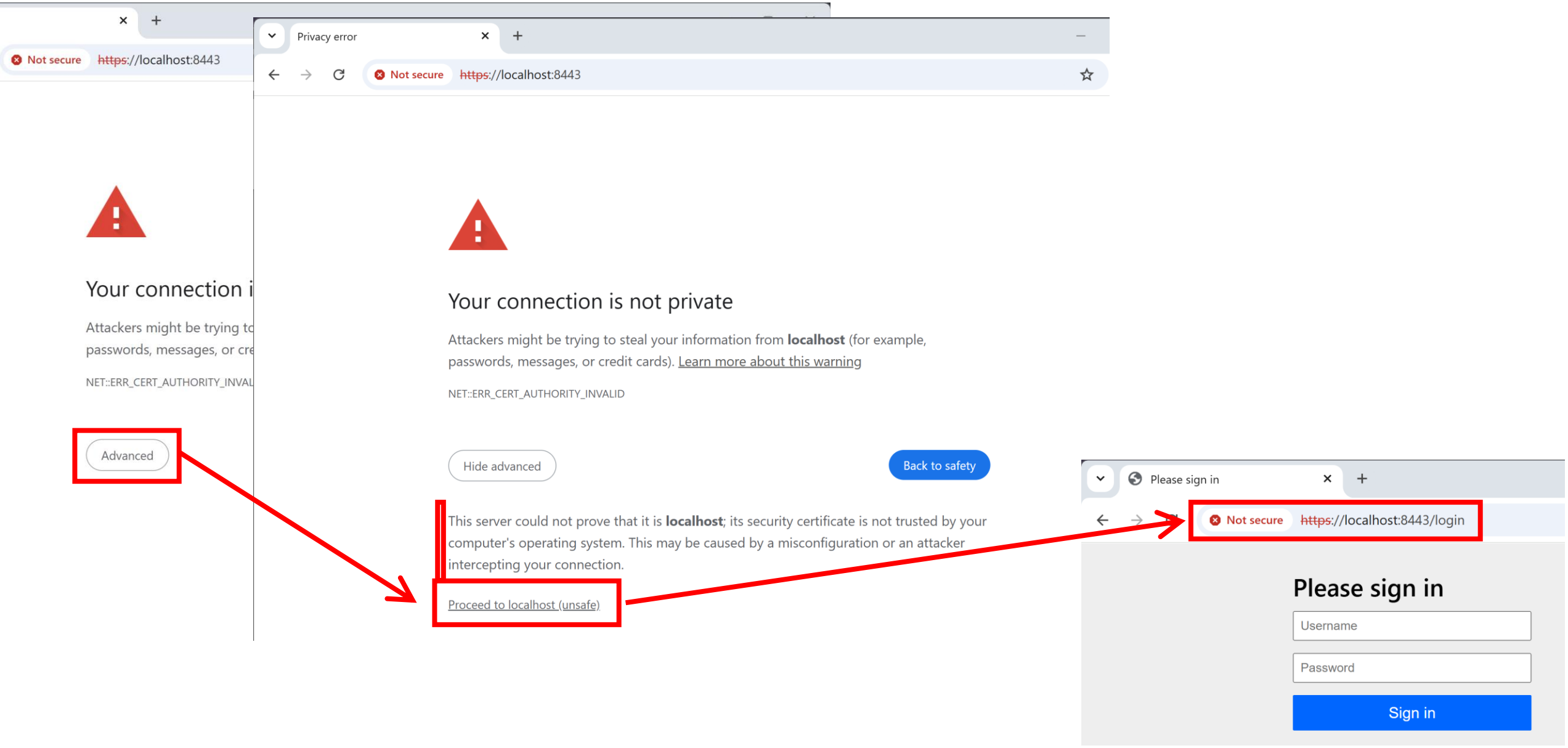
=> click on "ADVANCED" button to accept self-signed certificate

=> Add in Windows (CertMgr) User-defined certificates

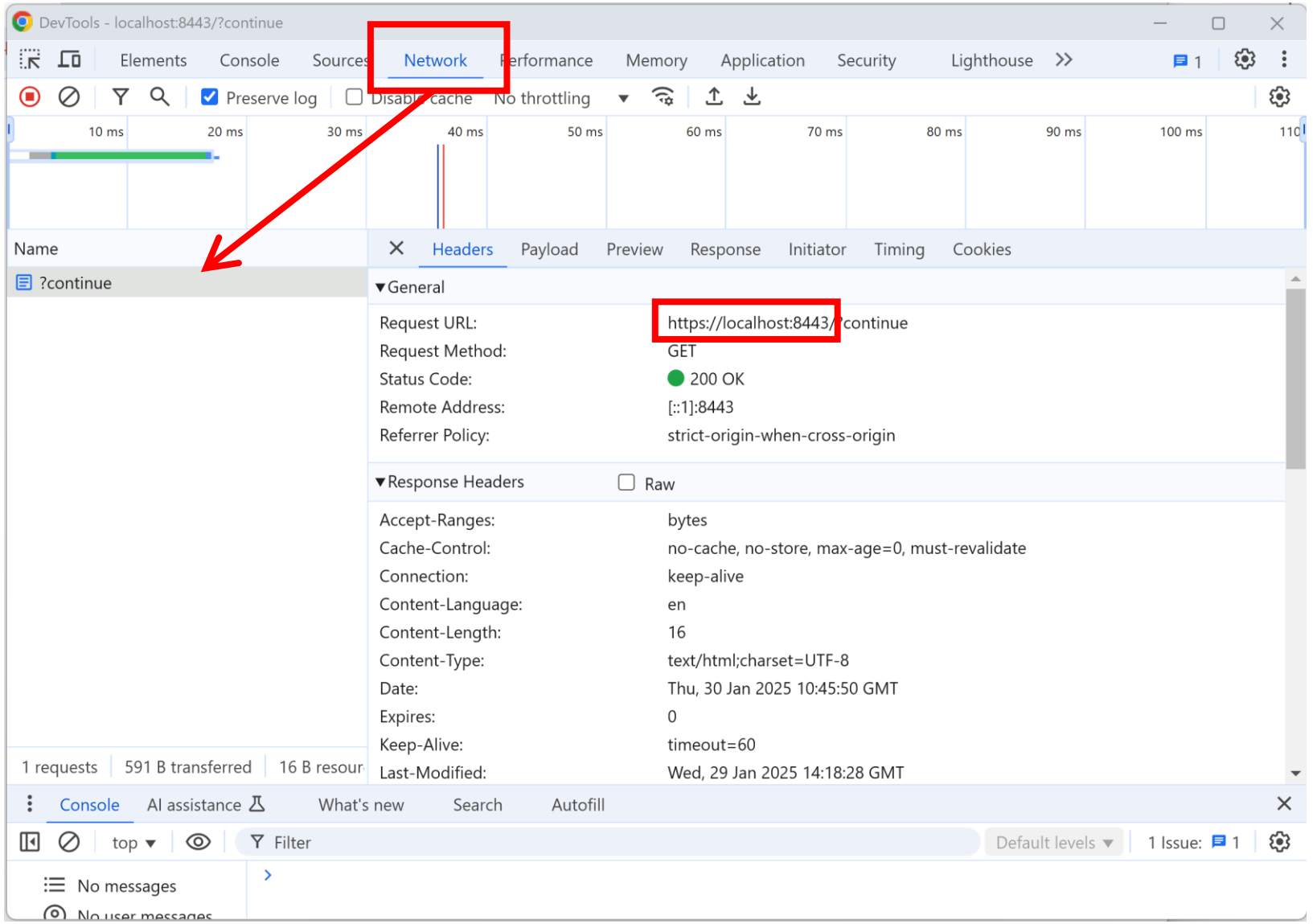
=> pay 200 EURO/year to have a valid certificate ?

=> use GO-DADDY or other signing Authority on internet (recognized by Chrome)

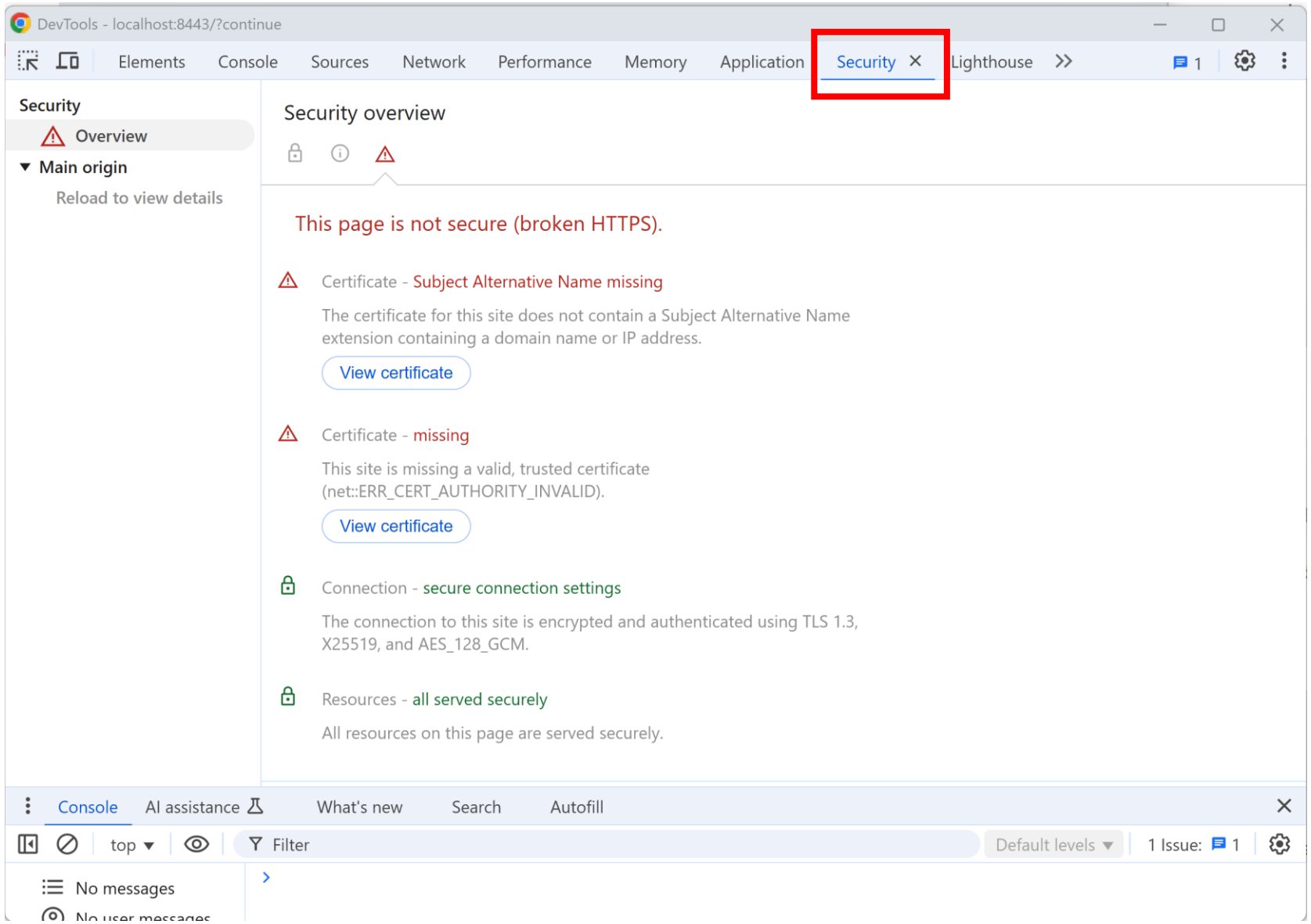
Advanced > Proceed to (unsafe)



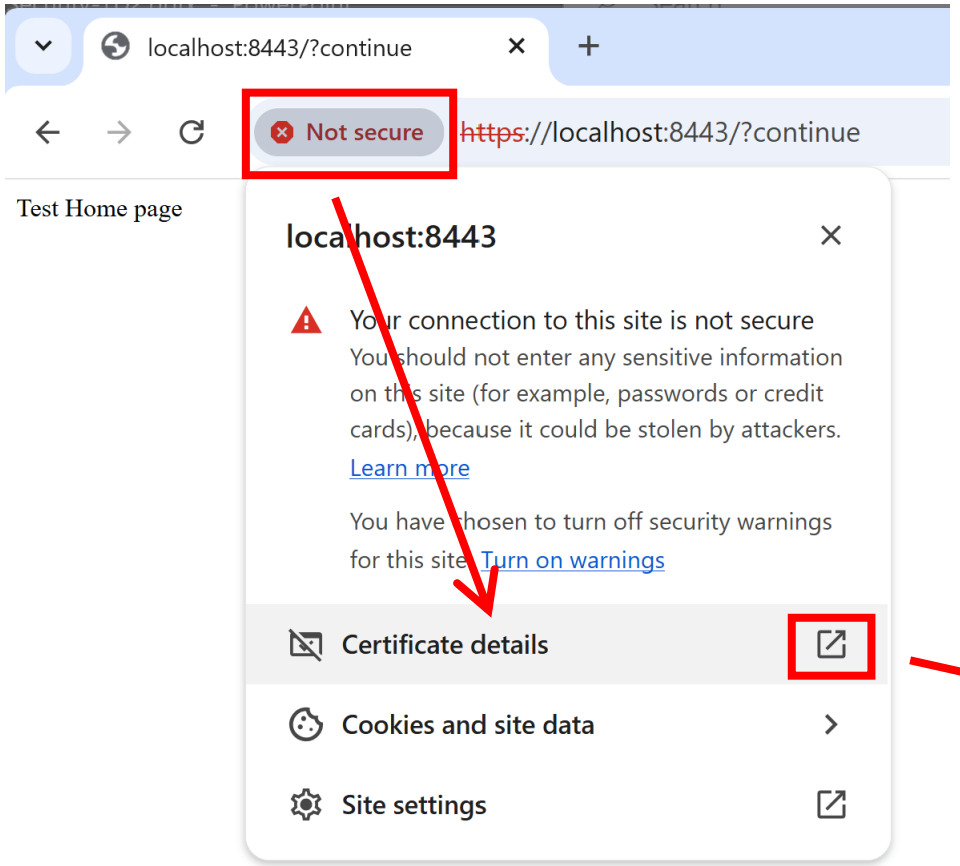
Chrome DevTools Network requests
... same on "https://"



Chrome DevTools "Security" tab

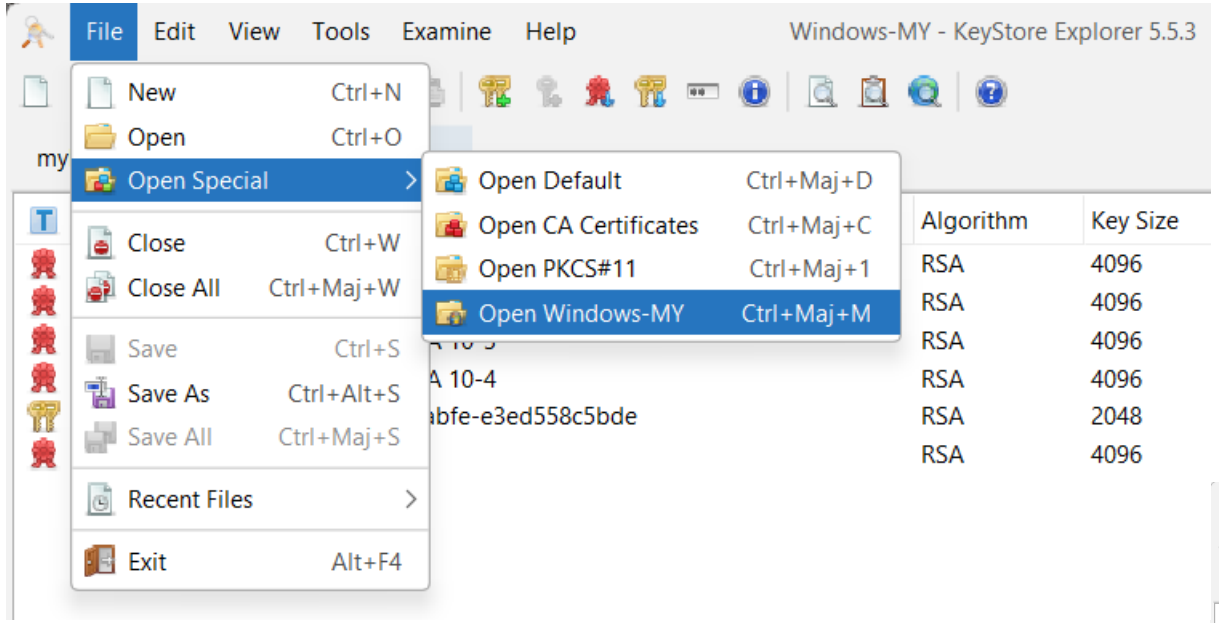


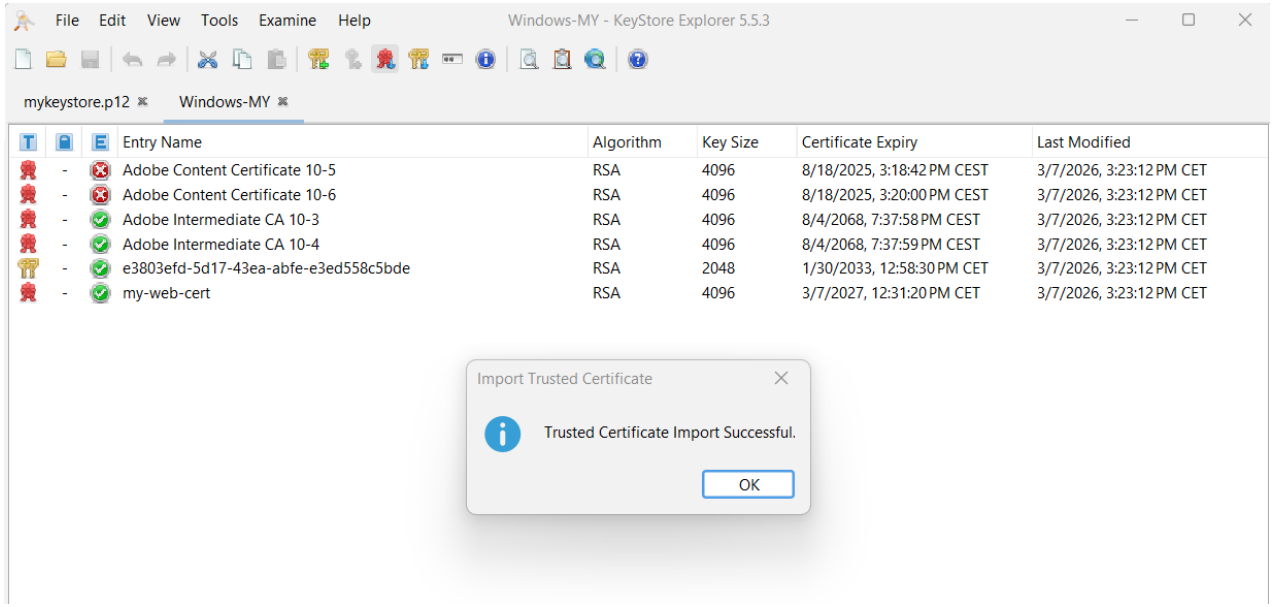
See Also Chrome -> Certificate Details



Suggestion ...
open view on a real web-site, "https://www.google.fr"
to see a valid signed certificate

Importing your Self-Signed Cert in Windows (Using KeyStore Explorer)



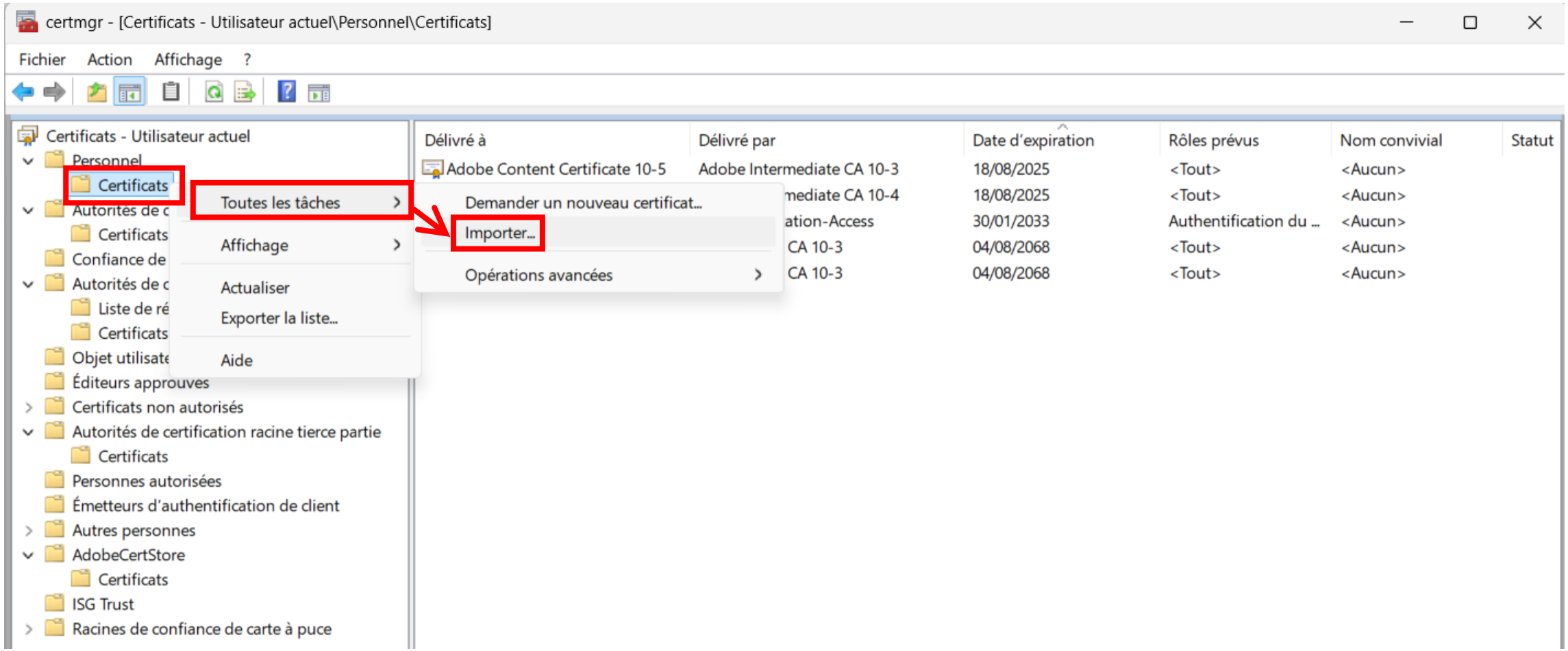


Alternative… Importing directly in Windows (20 Clicks)

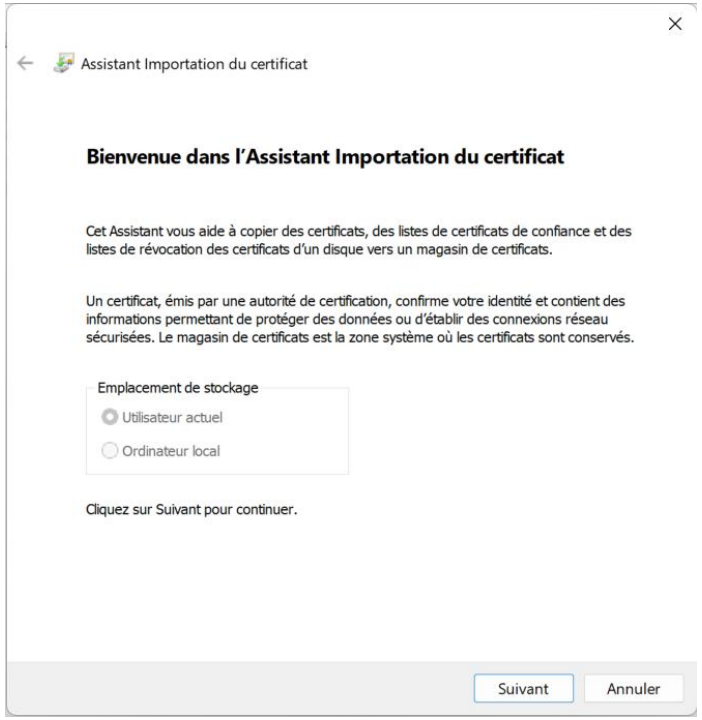
Notice old versions of Windows used to support only « *.pfx » file
(need to do conversion PEM / PKCS12 -> .pfx)

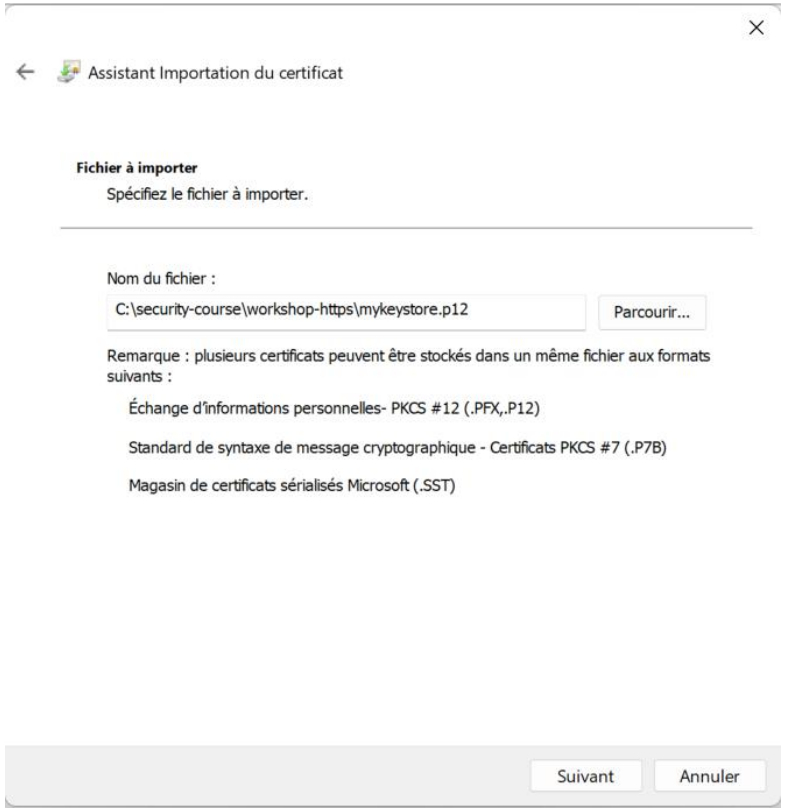
Now OK, can import directly from PKCS12   (but not jks)

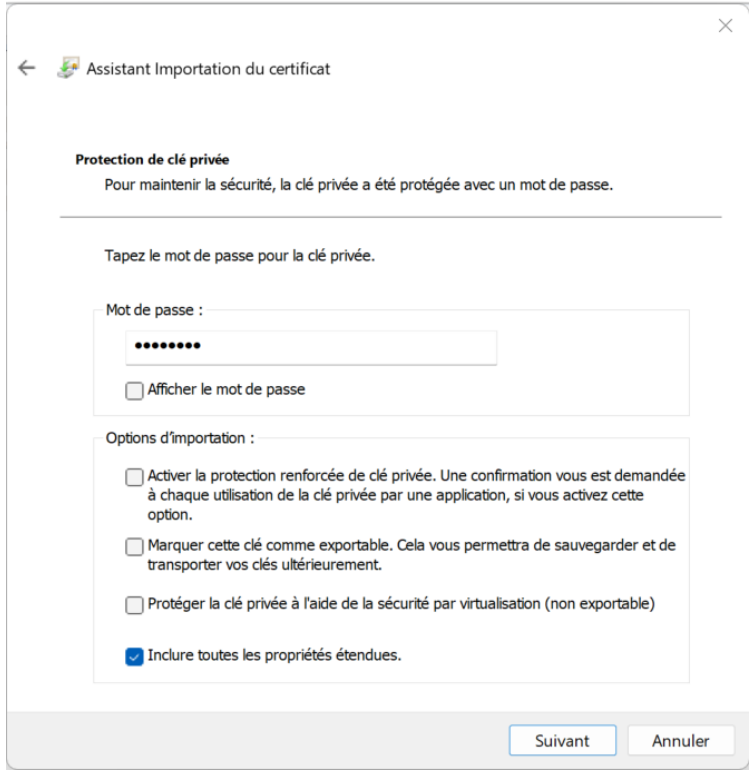
Importing Certificate in Windows



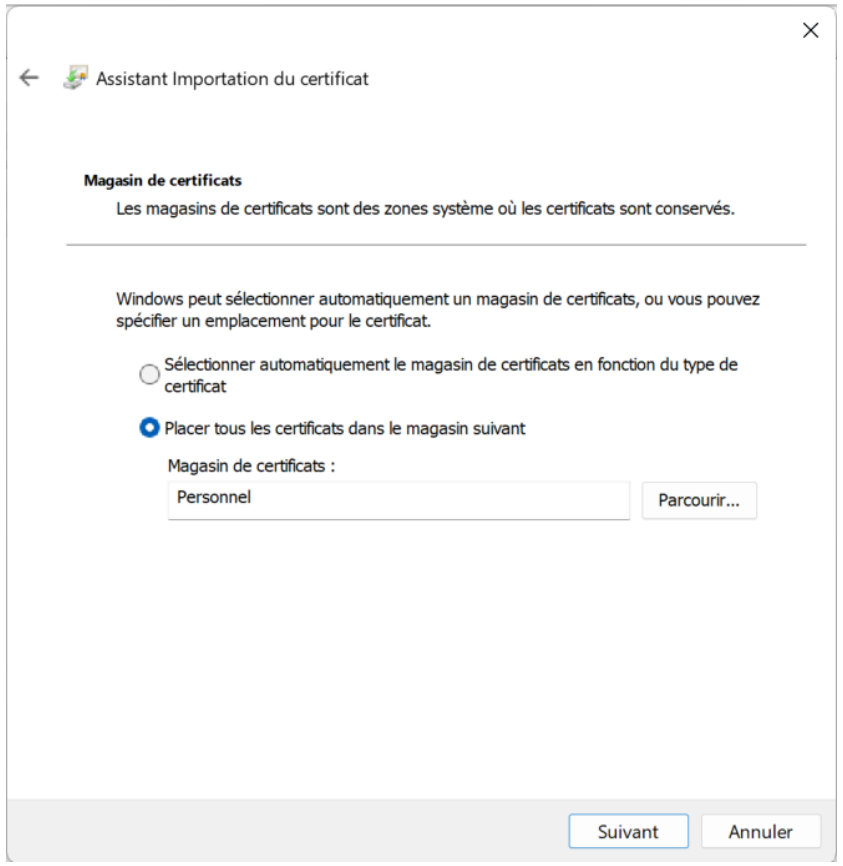
Importing …

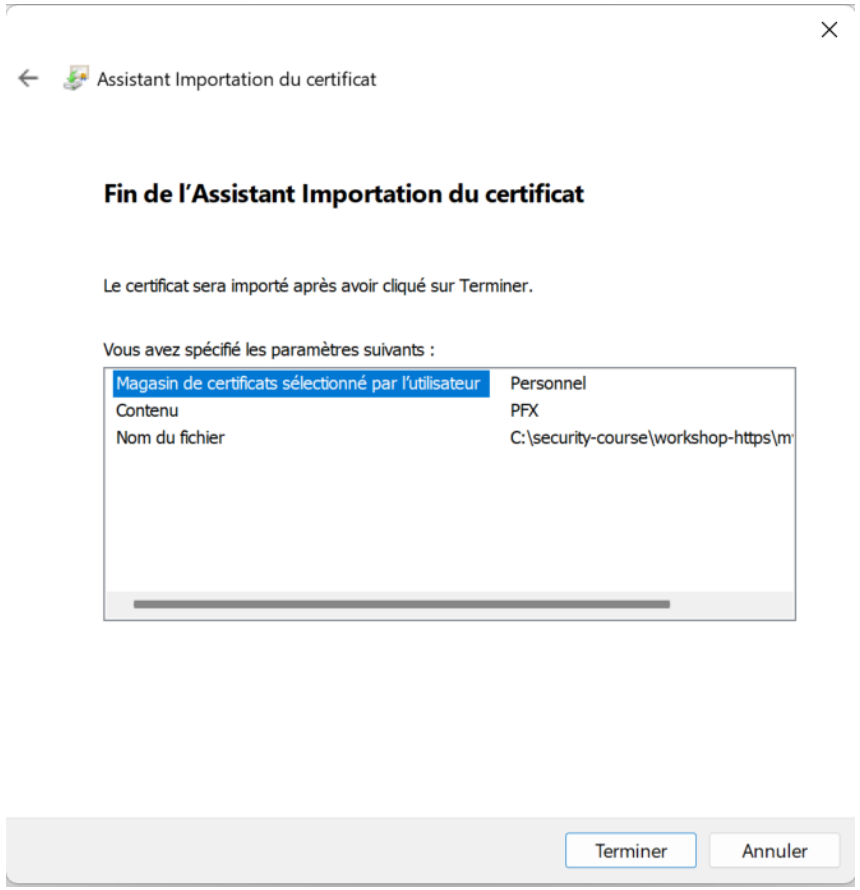

Assistant Importation du certificat
Bienvenue dans l'Assistant Importation du certificat
Cet Assistant vous aide à copier des certificats, des listes de certificats de confiance et des listes de révocation des certificats d'un disque vers un magasin de certificats.
Un certificat, émis par une autorité de certification, confirme votre identité et contient des informations permettant de protéger des données ou d'établir des connexions réseau sécurisées. Le magasin de certificats est la zone système où les certificats sont conservés.
Emplacement de stockage
Utilisateur actuel
Ordinateur local
Cliquez sur Suivant pour continuer.
Suivant
Annuler

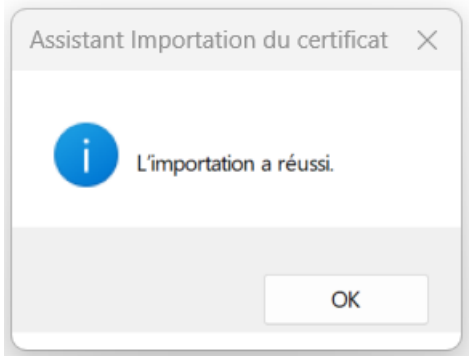

Assistant Importation du certificat
Fichier à importer
Spécifiez le fichier à importer.
Nom du fichier :
C:\security-course\workshop-https\mykeystore.p12
Parcourir...
Remarque : plusieurs certificats peuvent être stockés dans un même fichier aux formats suivants :
Échange d'informations personnelles- PKCS #12 (.PFX,.P12)
Standard de syntaxe de message cryptographique - Certificats PKCS #7 (.P7B)
Magasin de certificats sérialisés Microsoft (.SST)
Suivant
Annuler


Assistant Importation du certificat
Protection de clé privée
Pour maintenir la sécurité, la clé privée a été protégée avec un mot de passe.
Tapez le mot de passe pour la clé privée.
Mot de passe :
••••••••
Afficher le mot de passe
Options d'importation :
Activer la protection renforcée de clé privée. Une confirmation vous est demandée à chaque utilisation de la clé privée par une application, si vous activez cette option.
Marquer cette clé comme exportable. Cela vous permettra de sauvegarder et de transporter vos clés ultérieurement.
Protéger la clé privée à l'aide de la sécurité par virtualisation (non exportable)
Inclure toutes les propriétés étendues.
Suivant
Annuler

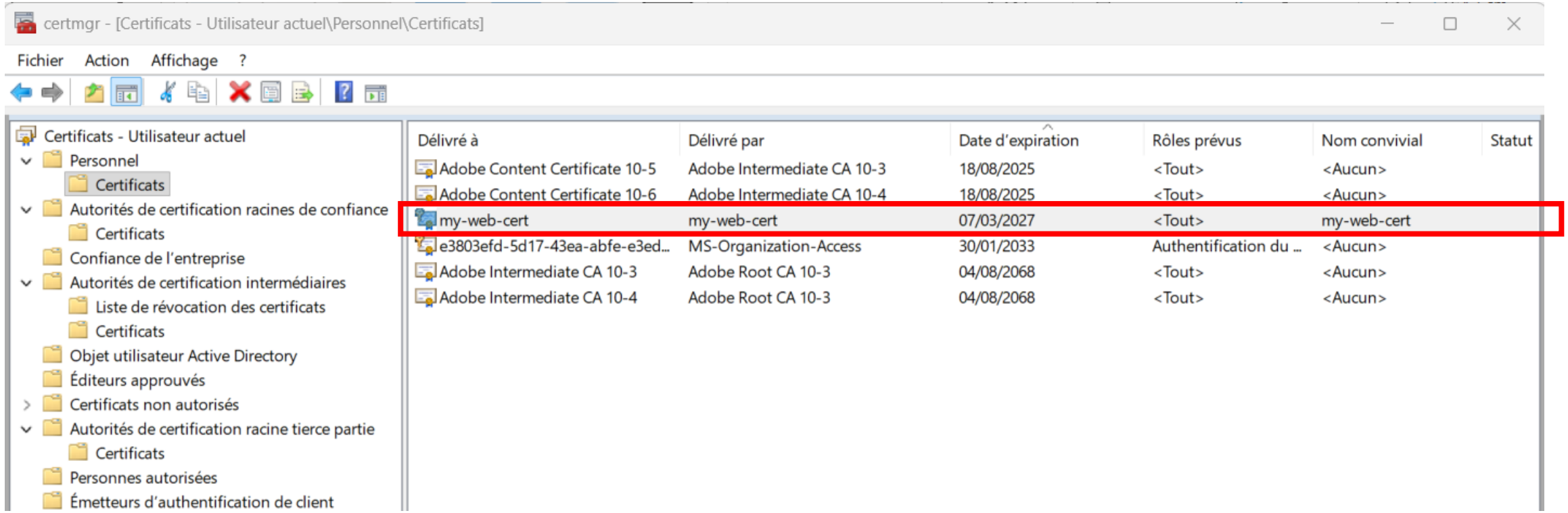
Importing…


Assistant Importation du certificat
Magasin de certificats
Les magasins de certificats sont des zones système où les certificats sont conservés.
Windows peut sélectionner automatiquement un magasin de certificats, ou vous pouvez spécifier un emplacement pour le certificat.
Sélectionner automatiquement le magasin de certificats en fonction du type de certificat
Placer tous les certificats dans le magasin suivant
Magasin de certificats :
Personnel
Parcourir...
Suivant
Annuler


Assistant Importation du certificat
Fin de l'Assistant Importation du certificat
Le certificat sera importé après avoir cliqué sur Terminer.
Vous avez spécifié les paramètres suivants :
Magasin de certificats sélectionné par l'utilisateur
Personnel
Contenu
PFX
Nom du fichier
C:\security-course\workshop-https\m
Terminer
Annuler


Assistant Importation du certificat
L'importation a réussi.
OK

Imported !


certmgr - [Certificats - Utilisateur actuel\Personnel\Certificats]
Fichier
Action
Affichage
?
Certificats - Utilisateur actuel
Personnel
Certificats
Autorités de certification racines de confiance
Certificats
Confiance de l'entreprise
Autorités de certification intermédiaires
Liste de révocation des certificats
Certificats
Objet utilisateur Active Directory
Éditeurs approuvés
Certificats non autorisés
Autorités de certification racine tierce partie
Certificats
Personnes autorisées
Émetteurs d'authentification de client
Délivré à
Délivré par
Date d'expiration
Rôles prévus
Nom convivial
Statut
Adobe Content Certificate 10-5
Adobe Intermediate CA 10-3
18/08/2025
<Tout>
<Aucun>
Adobe Content Certificate 10-6
Adobe Intermediate CA 10-4
18/08/2025
<Tout>
<Aucun>
my-web-cert
my-web-cert
07/03/2027
<Tout>
my-web-cert
e3803efd-5d17-43ea-abfe-e3ed...
MS-Organization-Access
30/01/2033
Authentification du ...
<Aucun>
Adobe Intermediate CA 10-3
Adobe Root CA 10-3
04/08/2068
<Tout>
<Aucun>
Adobe Intermediate CA 10-4
Adobe Root CA 10-3
04/08/2068
<Tout>
<Aucun>

… Check Your web page is now « Secured »

Relaunch your Web Browser (re-import Windows->Chrome??)

Re-open https://localhost:443

Check that Web site is « Secured » … take screenshot

Questions ?


Encryption
Openssl
Encryption
Certificate
https://
Tls
Tls
Sha256
https://
Openssl
Encryption
Encryption
Openssl
Certificate
Certificate
Cipher
Handshake
https://
X509
Handshake
Trust
Keypair
https://
Encryption
Openssl
Tls
Tls
Integrity
Trust
Openssl
X509
X509
Trust
Certificate
Handshake
Subject
PKCS12
https://
Openssl
Trust
Encryption
Openssl
Trust
Validation
Encryption
Tls
Tls
Openssl
Trust
Tls
Handshake
Tls
Tls
Certificate Chain
Trust
Encryption
Certificate
Certificate
Openssl
Security
X509
https://
Openssl
Authentication
https://
Openssl
X509
Chain
Handshake
Trust
Certificate
Encryption
Certificate
Openssl
Openssl
Certificate
Tls
Privatekey
Handshake
Trust
Openssl
X509
Publickey
Issuer
https://
Tls
Publickey
Rootca
Handshake
Certificate
X509
https://
Verification
Handshake
X509
Certificate
Handshake
X509